“MEMORY CARD””

PROYECTO DE DESARROLLO SOFTWARE

CICLO FORMATIVO DE GRADO SUPERIOR DE DESARROLLO DE APLICACIONES MULTIPLATAFORMA

ADRIÁN CID, ADRIÁN JUBERA Y RODRIGO SOSA
JAVIER GONZÁLEZ SANTURDE

Curso 2025-2026

ÍNDICE

1. Resumen. 3
2. Introducción. 5
3. Justificación y Objetivos. 6
4. Material y Método. 8
5. Resultados/Contenido. 10
6. Conclusiones y discusión. 14
7. Bibliografía. 15
8. Anexos. 16

1. Resumen.

Este Trabajo de Fin de Grado se detalla el proceso de desarrollo de la aplicación "Memory Card", un sistema de gestión, organización y red social relacionada con el mundo de los videojuegos.

El objetivo principal al que quiere aspirar este proyecto es ofrecer a los usuarios un espacio unificado e intuitivo donde puedan registrar, agrupar, valorar y reseñar los diferentes productos, dando solución al problema planteado de la excesiva cantidad de juegos repartidos entre las múltiples plataformas digitales y físicas, ya que esta situación conlleva un descontrol en el seguimiento de los títulos que los usuarios ya poseen.

Para darle solución a esta controversia, se ha planteado y ejecutado una arquitectura robusta, con un diseño amigable para que cualquier usuario pueda seguir el flujo de la aplicación y una seguridad excepcional residida en una base de datos relacional.

Utilizando la misma base de datos, se han construido desde cero dos entornos independientes: por un lado, una página web moderna, utilizando la librería React.js combinada con Vite; y, por otro lado, una aplicación móvil nativa para dispositivos Android programada en Java con un maquetado en XML. Estos dos productos comparten el mismo tipo de interfaz, pero con los cambios necesarios para los distintos dispositivos.

En cuanto al método de trabajo, este proyecto se ha desarrollado bajo una metodología de trabajo ágil basada en la asignación de Sprints semanales, garantizando un flujo de trabajo colaborativo mediante el uso de GitHub para el control de versiones, Figma para el prototipado de interfaces y Notion para la coordinación y asignación de tareas del equipo.

PALABRAS CLAVE: React, Android, XML, Sprints, GitHub, Figma, Notion.

This Final Degree Project details the development process of the "Memory Card" application, a management, organization, and social networking system related to the world of video games.

The main objective of this project is to offer users a unified and intuitive space where they can register, group, rate, and review different products, addressing the problem of the excessive number of games distributed across multiple digital and physical platforms. This situation leads to a lack of control in tracking the titles that users already own.

To resolve this issue, a robust architecture has been designed and implemented, with a user-friendly design so that any user can follow the application's flow, and exceptional security based on a relational database.

Using the same database, two independent environments have been built from scratch: a modern website using the React.js library combined with Vite; and a native

mobile application for Android devices programmed in Java with an XML layout. These two products share the same type of interface, but with the necessary modifications for different devices.

Regarding the workflow, this project was developed using an agile methodology based on weekly sprints, ensuring a collaborative workflow through the use of GitHub for version control, Figma for interface prototyping, and Notion for team coordination and task assignment.

KEY WORDS: React, Android, XML, Sprints, GitHub, Figma, Notion.

2. Introducción.

Hoy en día existe una gran cantidad de videojuegos y muchos de nosotros lo tenemos como hobbie principal. Para ello ha sido creado este pequeño proyecto llamado “Memory Card”.

Con esta aplicación queremos buscar un espacio en el que el usuario pueda registrar y organizar toda su colección de videojuegos, valorándolos, agruparlos y reseñarlos para llevar un recuento de todas las experiencias, a la vez que los juegos que tiene uno ganas de probar o los que están por terminar.

Con su diseño simple, limpio e intuitivo busca una experiencia de usuario adaptada a todo tipo de clientes para crear un flujo en la aplicación que sea sencillo de seguir. Con herramientas de filtrado no va a haber problema para buscar ese juego que se adapte a los gustos de cualquier persona

Uno de los puntos principales que nos ha llevado a desarrollar esta aplicación es que muchas veces no sabemos que juegos tenemos, ya sean por ofertas que compramos y luego no probamos o la colección que podemos comprar en Humble Bundle y luego se nos olvidan la mitad de los que vienen.

Decidimos seguir adelante con esta idea debido a que, en todos los miembros del grupo, este sector ha creado parte de nuestra personalidad y hemos crecido jugando, algo que es completamente necesario.

3. Justificación y Objetivos.

3.1. Justificación y motivos

Vemos que en la industria del videojuego no existe una aplicación ya consolidada donde usuarios puedan compartir de forma sencilla un seguimiento de lo que juegan y mucho menos, compartirlo con otros usuarios, a diferencia del cine, con la ya conocida y consolidada “Letterboxd”. Esto nos lleva a que personalmente, nuestros grupos de amigos tengan cada uno una aplicación o método diferente para llevar a cabo esa tarea, ya que ninguna de las existentes se use tanto en móvil como en web o sea lo suficientemente conocida por todos estos.

A nivel profesional, este Trabajo de Fin de Grado (TFG), es una gran oportunidad para simular un entorno de desarrollo de software real, que nos permita hacer frente a dos campos diferentes pero entrelazados que son el desarrollo web y el desarrollo de aplicaciones móviles. Al ser un grupo conformado por tres integrantes, nuestra mayor motivación es el reto de una integración de sistemas heterogénea; ya que no se trata de programar en tres campos diferentes (Java para móvil, React para Web y PHP para el backend), sino hacer que los tres estén coordinados para que la misma información se pueda compartir entre diferentes sistemas.

3.2. Objetivos del proyecto

3.2.1. Objetivo general

Diseñar, desarrollar y desplegar un sistema de información multiplataforma y distribuido para la gestión social de videojuegos, que permita a los usuarios llevar un registro personal, compartir información con otros usuarios y consultar información en tiempo real, manteniendo la consistencia de los datos entre una web y una aplicación móvil mediante un backend unificado y centralizado.

3.2.2. Objetivos específicos

Los objetivos específicos de este TFG están organizados en diferentes niveles:

A nivel organizativo:

- **Trabajar** con un flujo colaborativo basado en el control de versiones (GIT), usando metodología SCRUM en la asignación de tareas, para su fácil revisión e implementación.
- **Documentar** el desarrollo del software, incluyendo especificaciones para un mantenimiento y trabajo futuro, además de diagramas de flujo, casos de uso y entidad- relación.

A nivel técnico:

- **Diseñar** un modelo relacional (MySQL) escalable que soporte la gestión de los datos del proyecto.
- **Implementar** una **API RESTful** a través de PHP, definiendo *endpoints* claros y seguros para permitir la comunicación bidireccional (lectura y escritura) desde los clientes web y móvil.
- **Asegurar** la integridad y persistencia de la información, garantizando que las modificaciones realizadas en cualquiera de los clientes se reflejen inmediatamente en el otro.
- **Desarrollar** una aplicación nativa para dispositivos Android escrita en Java que aproveche los recursos del sistema, ofrezca una navegación fluida y se adapte a los elementos táctiles de los dispositivos móviles.
- **Crear** una interfaz web responsive usando React, priorizando la experiencia del usuario (UX) en la gestión de grandes índices de información.

A nivel de diseño:

- **Mostrar** una interfaz limpia y clara para guiar visualmente al usuario y mantener una interacción clara e intuitiva.
- **Diseñar** una aplicación con colores simples para resaltar la interfaz y los elementos visuales como carátulas y una tipografía adecuada a la temática.

4. Material y Método.

En este apartado detallamos tanto las herramientas técnicas que estamos usando como la forma en la que nos hemos organizado para trabajar en equipo.

4.1. Materiales (Recursos Técnicos)

Para montar todo el sistema de Memory Card, hemos seleccionado las siguientes tecnologías:

- Para el Backend: Estamos trabajando con PHP 8.x y un servidor Apache. La base de datos corre sobre MySQL, que es donde guardamos toda la información de usuarios y sus listas.
- Para la App Móvil: Usamos Android Studio y programamos en Java, aprovechando las herramientas del SDK de Android para que la aplicación sea nativa.
- Para la Web: Como entorno de desarrollo hemos elegido Visual Studio Code. Vamos a usar React.js junto con Vite. Básicamente, hemos elegido React porque nos facilita mucho organizar toda la interfaz por piezas y que la web se sienta "viva" cuando el usuario interactúa con ella. Usamos Vite porque, a la hora de programar, es mucho más rápido y ligero que lo que se usaba antes, así que nos ahorra tiempo y líos al montar el proyecto. Queremos que la experiencia de navegar por la colección de juegos sea más agradable.

Gestión y Diseño:

- Git y GitHub: Fundamental para no pisarnos el código y llevar un control de los cambios.
- Notion: Lo usamos como centro de organización en el que estarán definidas las tareas de cada uno, tendremos apartado de brainstorming, también ahí definiremos los SPRINTS y el tiempo que deberían durar.
- Figma: Aquí es donde hemos diseñado cómo se va a ver la aplicación y la web antes de ponernos a programar.
- Discord: Es la aplicación que hemos elegido para llevar a cabo nuestras propias reuniones de seguimiento.
- Draw.io: Es la web y extensión de Visual Studio Code que hemos usado para dibujar los diagramas.

4.2. Metodología del Proyecto

Nos hemos organizado con una metodología Ágil basada en SCRUM, dividida por Sprints. Al ser tres personas, esto nos permite dividir el trabajo en bloques, tener cada uno las tareas definidas y ver avances cada poco tiempo. El plan de trabajo sigue estos pasos:

1. **Análisis y Diseño:** Empezamos aclarando qué funciones queríamos (requisitos), montando el diagrama de la base de datos y haciendo los diseños en Figma.
2. **Desarrollo del Core (Backend):** Es el primer paso porque sin la base de datos y la API funcionando, ni la web ni la aplicación tendrían de dónde sacar la información.
3. **Desarrollo en Paralelo de los clientes:** Una vez que los *endpoints* (las rutas de la API) están listos, los encargados de **Android** y **React** pueden avanzar a la vez conectando las interfaces con los datos reales que vienen del backend.
4. **Integración y Pruebas:** Aquí es donde juntamos todo. Probamos que, si alguien añade un juego desde el móvil, aparezca al segundo en la web para asegurar que la información no se pierda.
5. **Documentación:** Realmente este paso se hace a la vez que cada uno de los anteriores.

- **Entorno de Ejecución:** Dejamos apuntado qué versiones de PHP y MySQL estamos usando para que, el día de la presentación, sepamos exactamente qué configuración necesita el servidor para que todo arranque sin fallos.
- **Protocolo de Pruebas:** No nos limitamos a programar; también hacemos un listado de pruebas (como crear usuarios de prueba o escribir reseñas) para confirmar que la comunicación entre el backend y los clientes es estable y no da errores.

5. Resultados/Contenido.

Tras realizar las fases de trabajo, descritas en la metodología, hemos logrado desarrollar un sistema de gestión de información distribuido y multiplataforma. A continuación, se detallan las conclusiones a las que hemos llegado siguiendo el orden cronológico de estas fases del proyecto "Memory Card".

5.1. Fase de Análisis y Diseño del Sistema

En esta primera fase, se definió tanto la estructura lógica como la identidad visual que sostiene el sistema de gestión. Los resultados se dividen en dos pilares fundamentales:

Arquitectura Funcional y de Datos

Se diseñó el modelo relacional con una arquitectura escalable para gestionar los datos de los usuarios y el catálogo global de videojuegos (**ver Diagrama 1 en Anexo 1**). Asimismo, se establecieron los flujos de interacción mediante diagramas de Casos de Uso (UML) (**ver Diagrama 2 en Anexo 1**). Para garantizar la robustez del sistema, cada caso de uso cuenta con su respectiva tabla de especificaciones, diagrama de actividad y diagrama de secuencia (**ver Anexo 2**), cubriendo:

- **Gestión de Accesos:** Inicio de sesión, registro, recuperación de contraseña y cierre de sesión (**ver Bloque A del Anexo 2**).
- **Gestión de Colección:** Búsqueda de títulos, adición a biblioteca personal y gestión de listas (**ver Bloque B del Anexo 2**).
- **Interacción Social:** Redacción de reseñas, sistema de valoraciones ("likes") y gestión de seguidores (**ver Bloque C del Anexo 2**).

Diseño Estético y Experiencia de Usuario (UX/UI)

El diseño visual se fundamentó en los principios de las **Leyes de UX**, buscando una interfaz intuitiva que minimice la carga cognitiva. Se aplicaron conceptos como las *Leyes de Gestalt* y otras para que el usuario se sienta familiarizado con la interfaz desde el primer uso.

- **Identidad Visual:** Se definió una **paleta de colores** común (**ver Imagen 1 en Anexo 3**) con tonos simples para resaltar los elementos visuales, como las carátulas de los juegos.

- **Prototipado:** Utilizando **Figma**, se crearon prototipos de alta fidelidad que permitieron validar el diseño y el flujo de navegación antes de la fase de programación (**ver Imágenes 3 y 4 en Anexo 3**).

5.2. Desarrollo del Core: Backend y Base de Datos

Siguiendo nuestra planificación, se implementó el "cerebro" central del proyecto sobre un servidor web **Apache**, configurado con **PHP 8.x** que actúa como el lenguaje de lado del servidor (backend) que procesa la lógica de negocio.

- **Persistencia de Datos:** Se construyó la base de datos en **MySQL**, su elección se debe a su alta fiabilidad para garantizar la integridad referencial de los datos. Nos ha permitido estructurar de forma eficiente las tablas para vincular los perfiles de usuario con el catálogo global de videojuegos, sus listas personalizadas y las valoraciones que emiten.

- **API RESTful:** Se desarrollaron los *endpoints* necesarios para recibir las peticiones HTTP de los clientes y procesar operaciones CRUD (Crear, Leer, Actualizar, Borrar) y devolver las respuestas en formato JSON. Esta API actúa como puente unificado, permitiendo que tanto el cliente web como el móvil consuman y modifiquen la misma información en tiempo real.

5.3. Desarrollo de Clientes: Web y Móvil

Una vez operativa la API, se procedió al desarrollo en paralelo de los entornos front-end.

5.3.1. Cliente Web Front-end (React)

Para el desarrollo de la plataforma accesible desde navegadores, hemos apostado por l**as siguientes tecnologías**, orientadas a la fluidez y modularidad**:**

- **React.js:** Es una biblioteca de JavaScript diseñada para construir interfaces de usuario basándose en componentes reutilizables.

- **Vite:** Para complementar a React, hemos utilizado Vite como herramienta de construcción (*build tool*) y servidor de desarrollo local. A diferencia de empaquetadores más antiguos, Vite nos ha proporcionado tiempos de carga en desarrollo casi instantáneos y una optimización superior en el empaquetado final para producción, agilizando enormemente nuestro flujo de trabajo.

El resultado es una **SPA (Single Page Application)** que ofrece una navegación responsive fluida por el catálogo sin recargas de página, optimizando la experiencia del usuario al gestionar grandes volúmenes de información (**ver Imágenes de 1 a 6 del Anexo 4**).

5.3.2. Aplicación Móvil Nativa (Android)

Se ha creado una aplicación que aprovecha los recursos nativos del sistema mediante el uso de estas tecnologías:

- **Android Studio y Java:** Hemos utilizado el entorno de desarrollo integrado (IDE) oficial de Google, Android Studio. Separando el desarrollo en dos pilares principales, el maquetado está escrito en **XML** al contrario que toda la parte lógica que está hecha en **Java**. Para conseguir esta última parte tuvimos que hacer downgrade de la versión ya que en las más modernas se programa en **Kotlin**.
- Android Studio tiene un sistema de emulación integrado utilizando SDK de dispositivos Android reales, gracias a esto pudimos acceder directamente a los recursos de hardware y a los componentes de interfaz del sistema operativo. Esto garantiza la correcta implementación del maquetado planteado al principio del desarrollo, las transiciones entre las distintas pantallas y la respuesta a los gestos táctiles que el usuario realiza (**ver Imágenes de 7 a 14 del Anexo 4**).

5.4. Integración, Pruebas y Sincronización

El resultado más crítico y satisfactorio de este proyecto ha sido la validación de la concurrencia de datos trabajando con estas tecnologías en conjunto. Durante las

pruebas de integración, confirmamos que al registrar una nueva reseña o añadir un juego a la lista de "pendientes" desde el cliente Android programado en Java, el cambio se refleja instantáneamente al recargar la vista en la interfaz web construida en **React**. Esto confirma que nuestra API en PHP funciona correctamente como puente unificado, cumpliendo el objetivo de mantener la consistencia en la colección del usuario sin importar el dispositivo que utilice.

6. Conclusiones y discusión.

Después de haber planteado y diseñado Memory Card, estas son nuestras conclusiones principales:

6.1. Aportación del proyecto

Creemos que la mayor ventaja de nuestro trabajo es que resolvemos un problema real para mucha gente que, como nosotros, tiene juegos repartidos por mil plataformas. Haber diseñado un sistema que une una aplicación móvil y una Web bajo un mismo "cerebro" (el backend) permite que el usuario tenga un seguimiento de su colección siempre a mano. Al igual que “Letterboxd” cambió la forma de registrar el cine, pensamos que Memory Card puede hacer lo mismo con los videojuegos, ofreciendo un sitio cómodo donde llevar el diario de lo que jugamos.

6.2. Reflexión técnica (Discusión)

Durante el desarrollo, nos hemos dado cuenta de que lo más difícil no es programar cada parte por separado, sino hacer que se entiendan entre ellas. Hemos discutido mucho sobre cómo organizar la base de datos para que sea rápida y también nos obliga a que nuestro backend sea muy sólido para que no haya fallos de sincronización. Estamos convencidos de que usar PHP y una API REST ha sido la mejor decisión porque nos ha dado la flexibilidad que necesitábamos.

A la hora de abarcar el desarrollo móvil, aprender un nuevo entorno nos ha enseñado que limitaciones tienen cada uno y que ventajas nos proporcionan otros, gracias a ello nos conseguimos adaptarnos a cualquiera que en un futuro tengamos que utilizar.

6.3. Relación con lo aprendido en el Grado

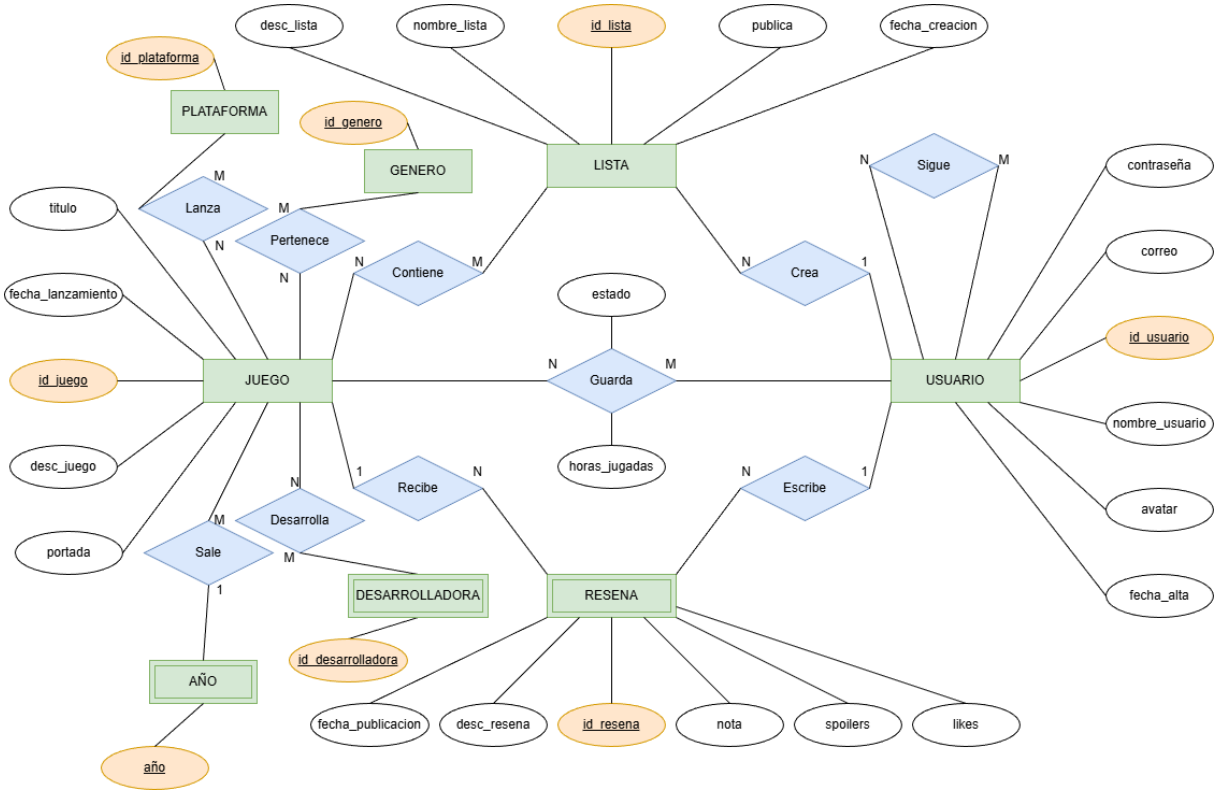
Este TFG nos ha servido para juntar todo lo que hemos visto en el ciclo de DAM. No es solo "hacer una aplicación", es gestionar el control de versiones con Git, organizar las tareas en Notion y entender cómo viajan los datos desde el servidor hasta el móvil del usuario. A su vez, estar en un desarrollo completo nos ayuda a interiorizar todos los componentes de una aplicación, ya sea web como móvil, que posee y como se relacionan entre ellos, ya sea con conexiones utilizando API como implementar una base de datos a un frontend. Debido al conjunto completo, nos ha dado una visión real de cómo se trabaja en una empresa de desarrollo.

7. Bibliografía.

- Android Developers. (2023). Guía de arquitectura de apps. Google. Recuperado de https://developer.android.com/topic/architecture
- Meta Open Source. (2024). React Documentation: Describing the UI. Recuperado de https://react.dev/learn
- MySQL. (2023). MySQL 8.0 Reference Manual. Oracle Corporation. Recuperado de https://dev.mysql.com/doc/refman/8.0/en/
- NPM. (2023). *About npm*. npm Docs. Recuperado de https://docs.npmjs.com/
- Schwaber, K., & Sutherland, J. (2020). La Guía de Scrum. Scrum.org. Recuperado de https://scrumguides.org/
- Vite. (2024). Next Generation Frontend Tooling. Recuperado de https://vitejs.dev/guide/
- Visual Paradigm. (2023). *Guía de aprendizaje de UML*. Visual Paradigm. https://www.visual-paradigm.com/es/guide/uml-unified-modeling-language/
- Yablonski, J. (s.f.). *Leyes de UX*. Recuperado el 13 de marzo de 2026, de https://lawsofux.com/es/
- ESDESIGN Barcelona. (2026, 2 de febrero). *Teoría de la Gestalt: leyes clave aplicadas al diseño visual*. https://www.esdesignbarcelona.com/actualidad/diseno-grafico/teoria-gestalt
- World Wide Web Consortium (W3C). (2023). *Web Content Accessibility Guidelines (WCAG) 2.2 Overview*. https://www.w3.org/WAI/standards-guidelines/wcag/
- JGraph. (s.f.). *UML diagrams in draw.io*. diagrams.net. Recuperado el 13 de marzo de 2026, de https://www.diagrams.net/blog/uml-diagrams

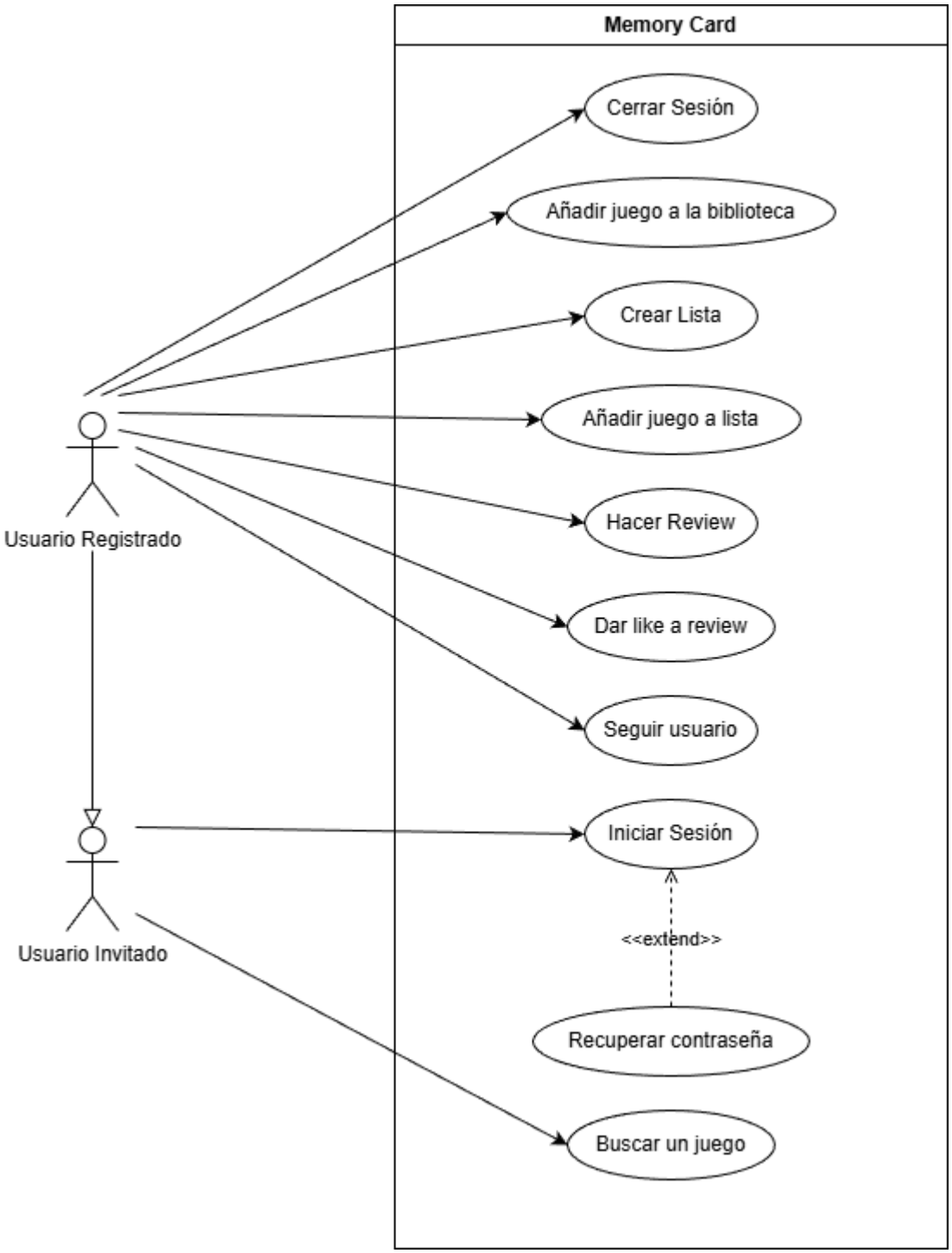
8. Anexos

Anexo 1. Arquitectura de Datos y Funcional



- **Diagrama 1. Diagrama Entidad-Relación de la Base de Datos (MySQL).**

Descripción: Representación del modelo relacional que soporta la persistencia de usuarios, videojuegos y relaciones sociales.



- **Diagrama 2. Mapa General de Casos de Uso (UML).**

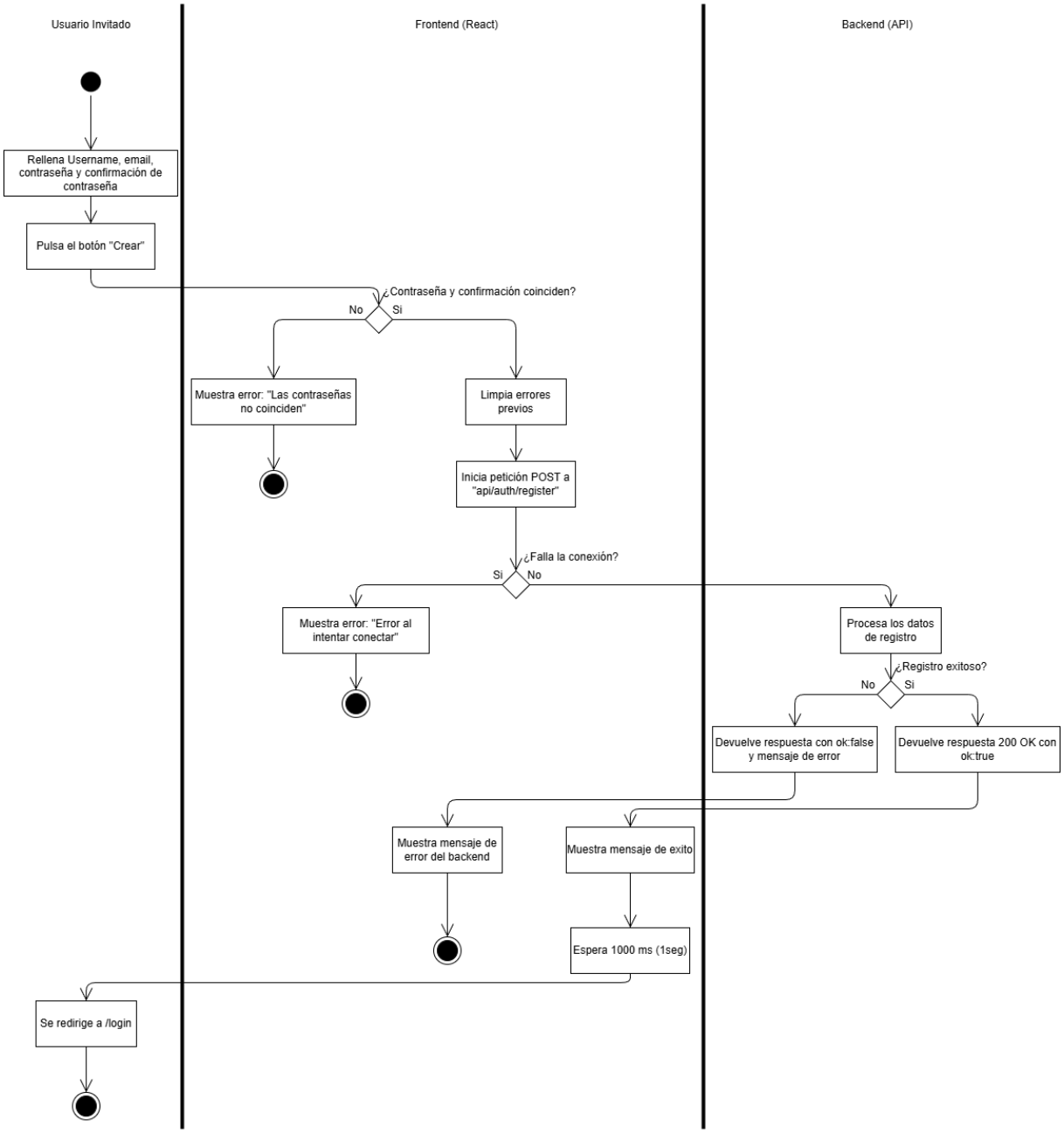
Descripción: Visión global de las principales funcionalidades y actores que interactúan con el ecosistema Memory Card.

Anexo 2. Especificación Técnica de Funcionalidades

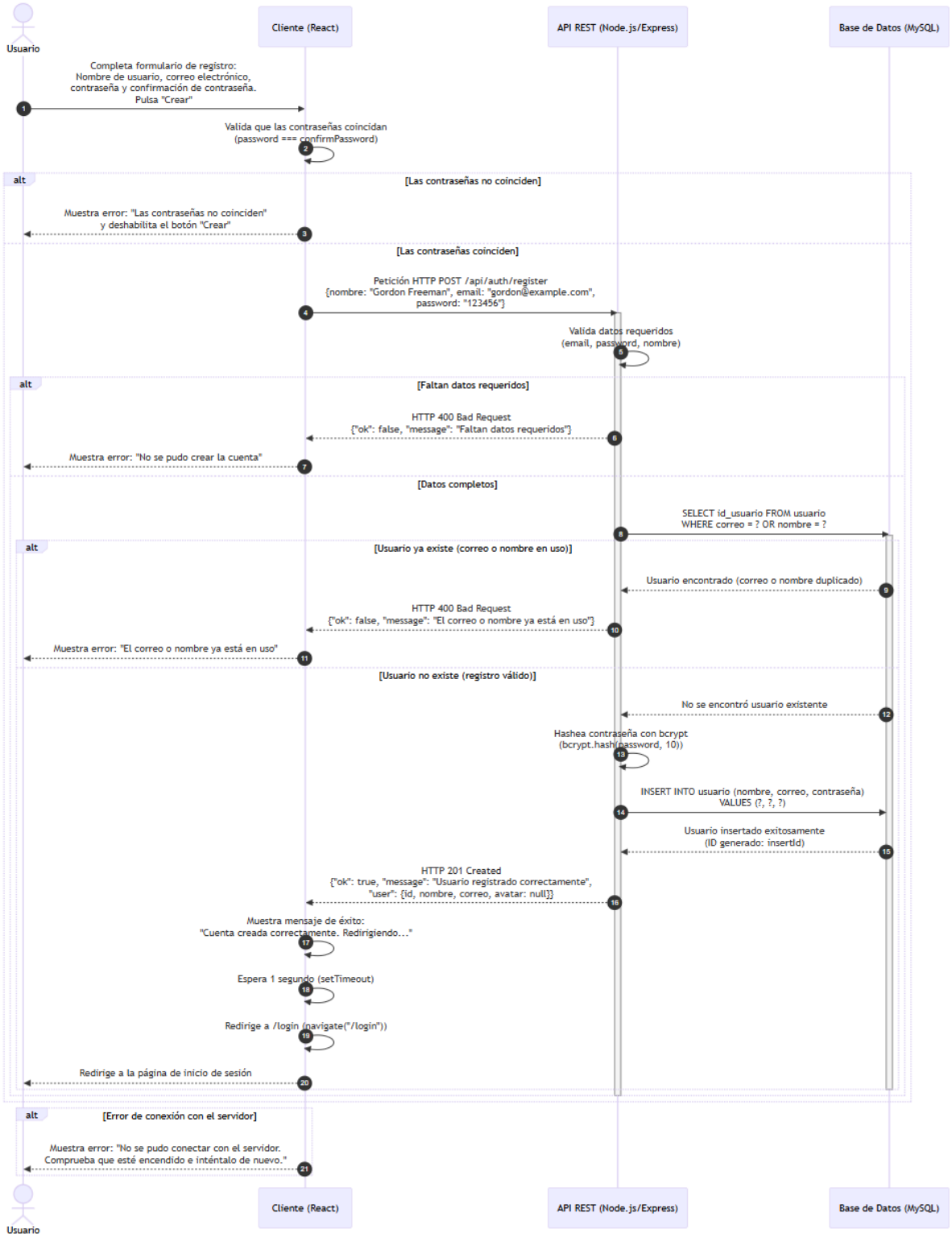
Este anexo contiene las **Tablas de Especificación**, los **Diagramas de Actividad** y los **Diagramas de Secuencia** para cada uno de los 11 casos de uso definidos.

Bloque A: Gestión de Accesos

- **Diagramas 1 y 2 y Tabla 1.** Registro de usuario.



- **Diagrama 1.** Diagrama de Actividad referente al registro de Usuarios.

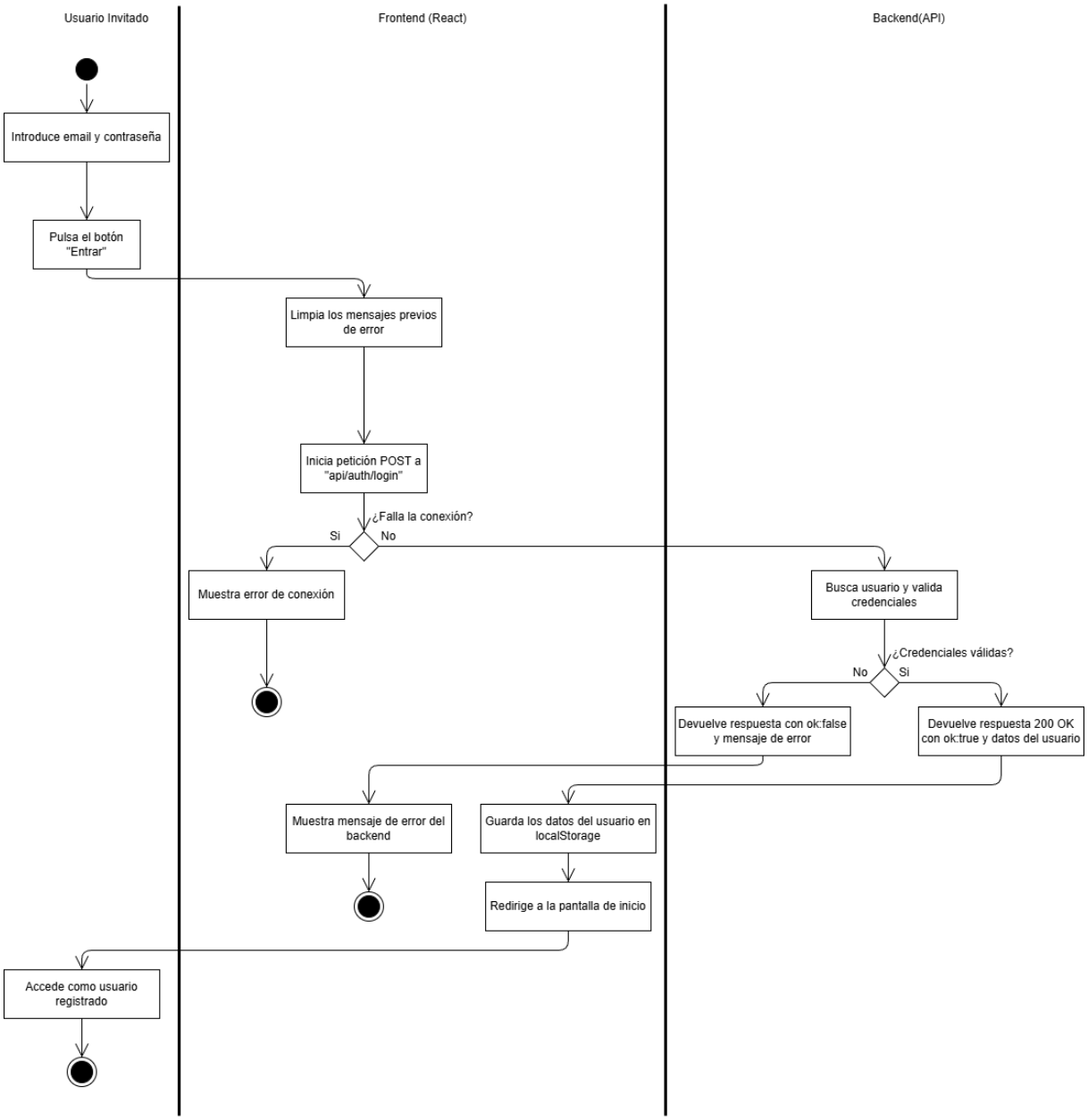


- **Diagrama 2.** Diagrama de Secuencia referente al registro de Usuarios

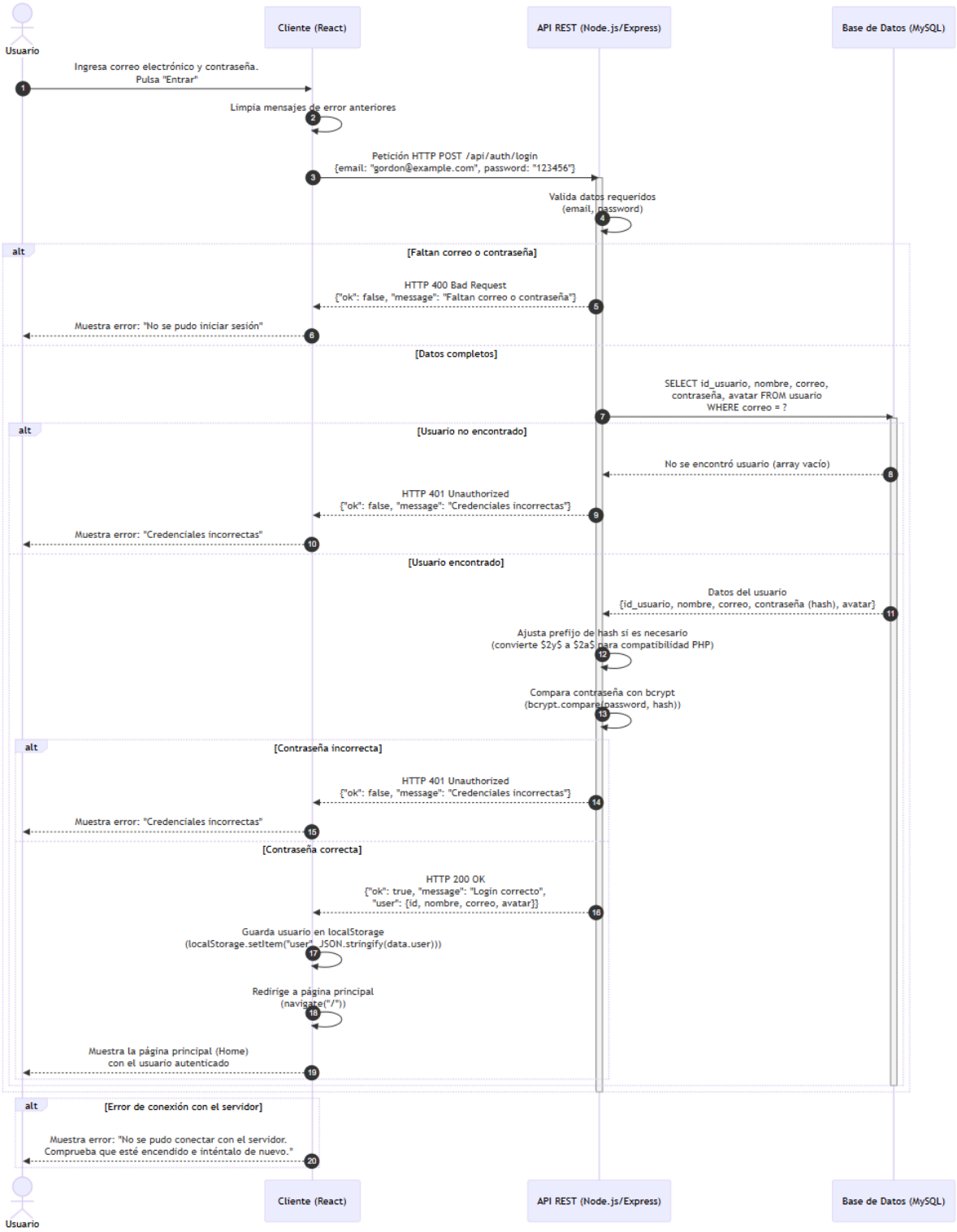
NOMBRE DEL CASO	Registrar nuevo usuario
DESCRIPCION	Un usuario no registrado previamente rellena el formulario para crear una cuenta
ACTORES	Usuario Invitado
PRECONDICIONES	El usuario no debe tener una cuenta creada ni una sesión iniciada
SECUENCIA NORMAL	1. El usuario introduce nombre de usuario, correo, contraseña y confirmación de contraseña. 2. El usuario pulsa el botón "Crear". 3. El sistema valida localmente que ambas contraseñas son idénticas. 4. El sistema envía los datos al servidor. 5. El servidor registra al usuario con éxito. 6. El sistema muestra un mensaje de éxito ("Cuenta creada correctamente"). 7. Tras 1 segundo, el sistema redirige al usuario a la pantalla de Inicio de Sesión.
POSTCONDICIONES	El usuario queda registrado en la base de datos
EXCEPCIONES	- **Contraseñas distintas:** El botón se deshabilita y/o el sistema muestra el error *"Las contraseñas no coinciden"*. No se llama al servidor. - **Fallo del servidor/Datos inválidos:** Si la API devuelve un error (ej. email ya en uso), se muestra el mensaje devuelto por el servidor o *"No se pudo crear la cuenta"*. - **Caída de red:** Si falla la petición fetch, se muestra *"Error de red al intentar conectar con el servidor"*.

- **Tabla 1.** Tabla de especificación de requisitos referente al registro de Usuarios.

- **Diagramas 3 y 4 y Tabla 2.** Inicio de sesión.



- **Diagrama 3.** Diagrama de Actividad referente al Inicio de Sesión.



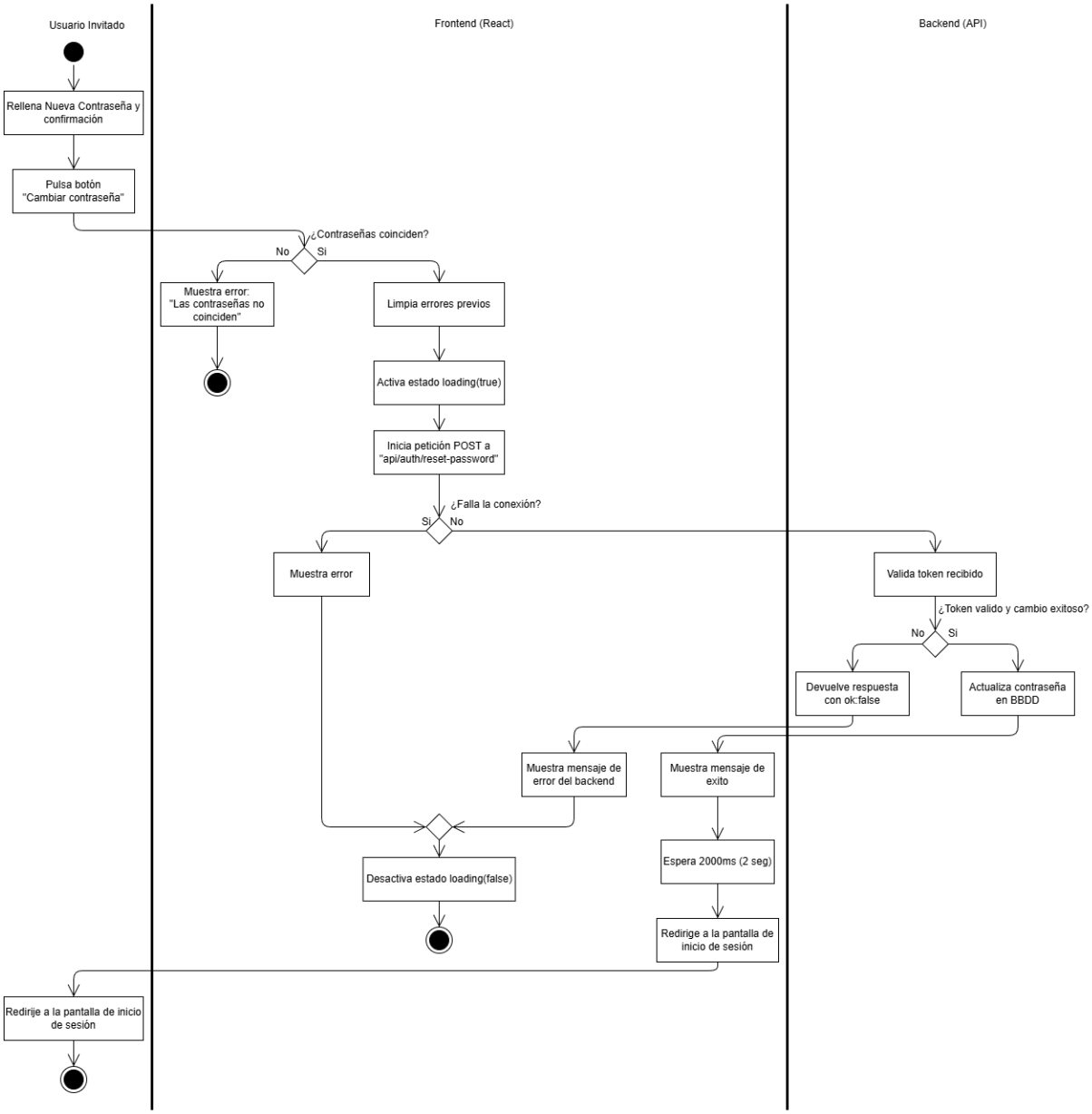
- **Diagrama 4.** Diagrama de Secuencia referente al Inicio de Sesión.

NOMBRE DEL CASO	Iniciar sesión
DESCRIPCION	Un usuario ya registrado accede a su cuenta rellenado el formulario

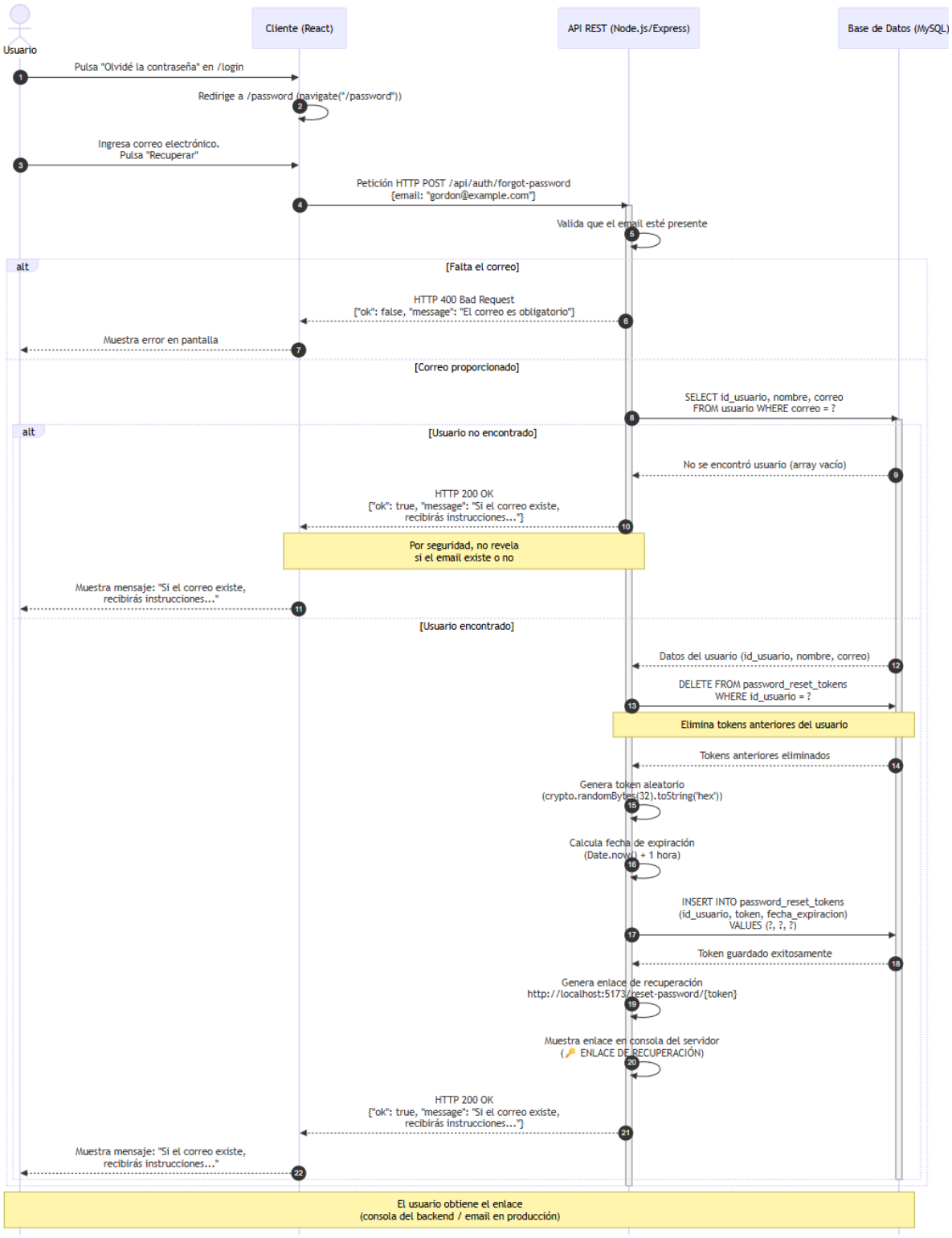
ACTORES	Usuario Invitado
PRECONDICIONES	El usuario debe estar registrado en la BBDD y no tener activa la sesión
SECUENCIA NORMAL	1. El usuario introduce su correo y contraseña. 2. El usuario pulsa el botón "Entrar". 3. El sistema limpia errores previos e inicia la petición al servidor. 4. El servidor valida las credenciales en la base de datos. 5. El servidor devuelve los datos del usuario. 6. El sistema guarda los datos del usuario en el almacenamiento local. 7. El sistema redirige al usuario a la página de Inicio.
POSTCONDICIONES	El usuario tiene una sesión activa y se encuentra en la pantalla principal.
EXCEPCIONES	- **Credenciales incorrectas:** El servidor devuelve un error. El sistema muestra el mensaje de error del backend o *"No se pudo iniciar sesión"*. No se guarda nada en el almacenamiento. - **Caída de red:** Si la conexión falla (entra al bloque catch), se muestra *"Error de red al intentar conectar con el servidor"*.

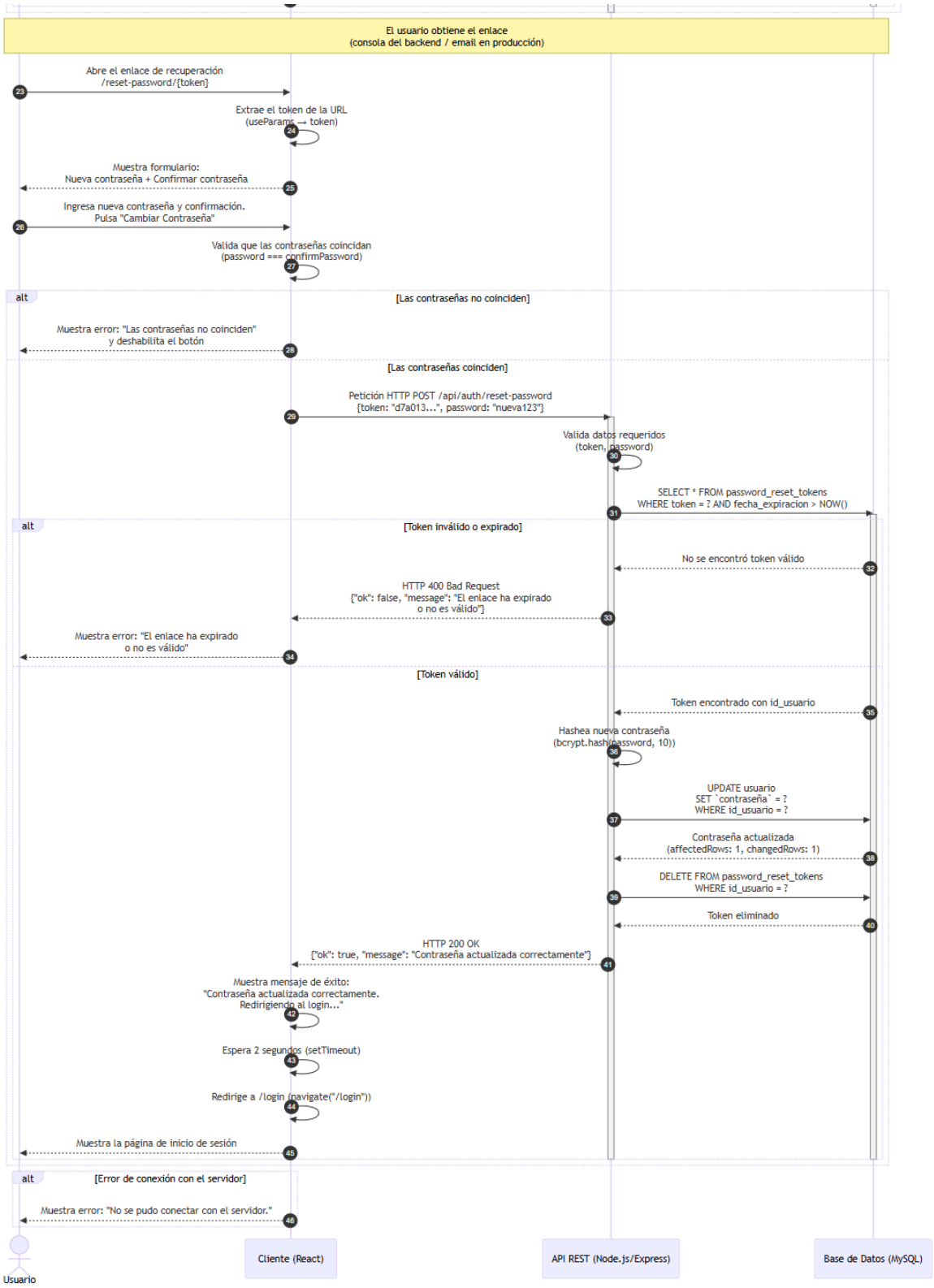
- **Tabla 2.** Tabla de especificación de requisitos referente al Inicio de Sesión.

- **Diagramas 5 y 6 y Tabla 3.** Recuperación de contraseña.



- **Diagrama 5.** Diagrama de Actividad referente a la recuperación de contraseña.


Usuario
Cliente (React)
API REST (Node.js/Express)
Base de Datos (MySQL)
1 Pulsa "Olvidé la contraseña" en /login
2 Redirige a /password (navigate("/password"))
3 Ingresa correo electrónico. Pulsa "Recuperar"
4 Petición HTTP POST /api/auth/forgot-password {email: "gordon@example.com"}
5 Valida que el email esté presente
alt
[Falta el correo]
6 HTTP 400 Bad Request {"ok": false, "message": "El correo es obligatorio"}
7 Muestra error en pantalla
[Correo proporcionado]
8 SELECT id_usuario, nombre, correo FROM usuario WHERE correo = ?
alt
[Usuario no encontrado]
9 No se encontró usuario (array vacío)
10 HTTP 200 OK {"ok": true, "message": "Si el correo existe, recibirás instrucciones..."}
Por seguridad, no revela si el email existe o no
11 Muestra mensaje: "Si el correo existe, recibirás instrucciones..."
[Usuario encontrado]
12 Datos del usuario (id_usuario, nombre, correo)
13 DELETE FROM password_reset_tokens WHERE id_usuario = ?
Elimina tokens anteriores del usuario
14 Tokens anteriores eliminados
15 Genera token aleatorio (crypto.randomBytes(32).toString('hex'))
16 Calcula fecha de expiración (Date.now() + 1 hora)
17 INSERT INTO password_reset_tokens (id_usuario, token, fecha_expiracion) VALUES (?, ?, ?)
18 Token guardado exitosamente
19 Genera enlace de recuperación http://localhost:5173/reset-password/{token}
20 Muestra enlace en consola del servidor (🔑 ENLACE DE RECUPERACIÓN)
21 HTTP 200 OK {"ok": true, "message": "Si el correo existe, recibirás instrucciones..."}
22 Muestra mensaje: "Si el correo existe, recibirás instrucciones..."
El usuario obtiene el enlace (consola del backend / email en producción)

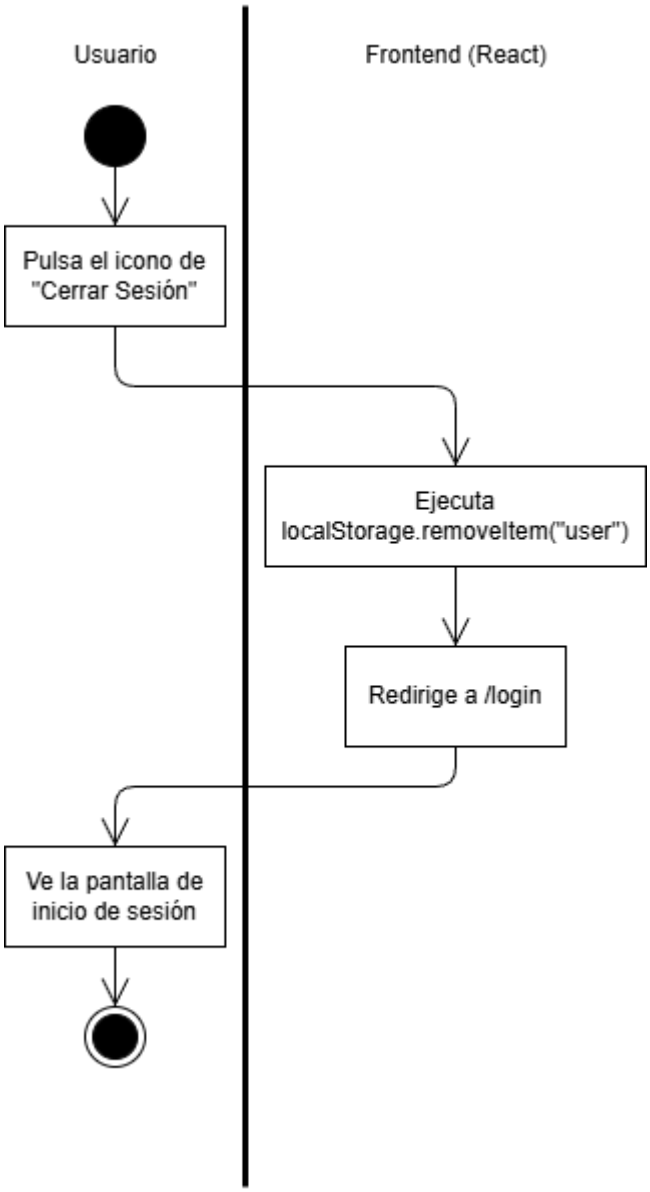


- **Diagrama 6.** Diagrama de Secuencia referente a la recuperación de contraseña.

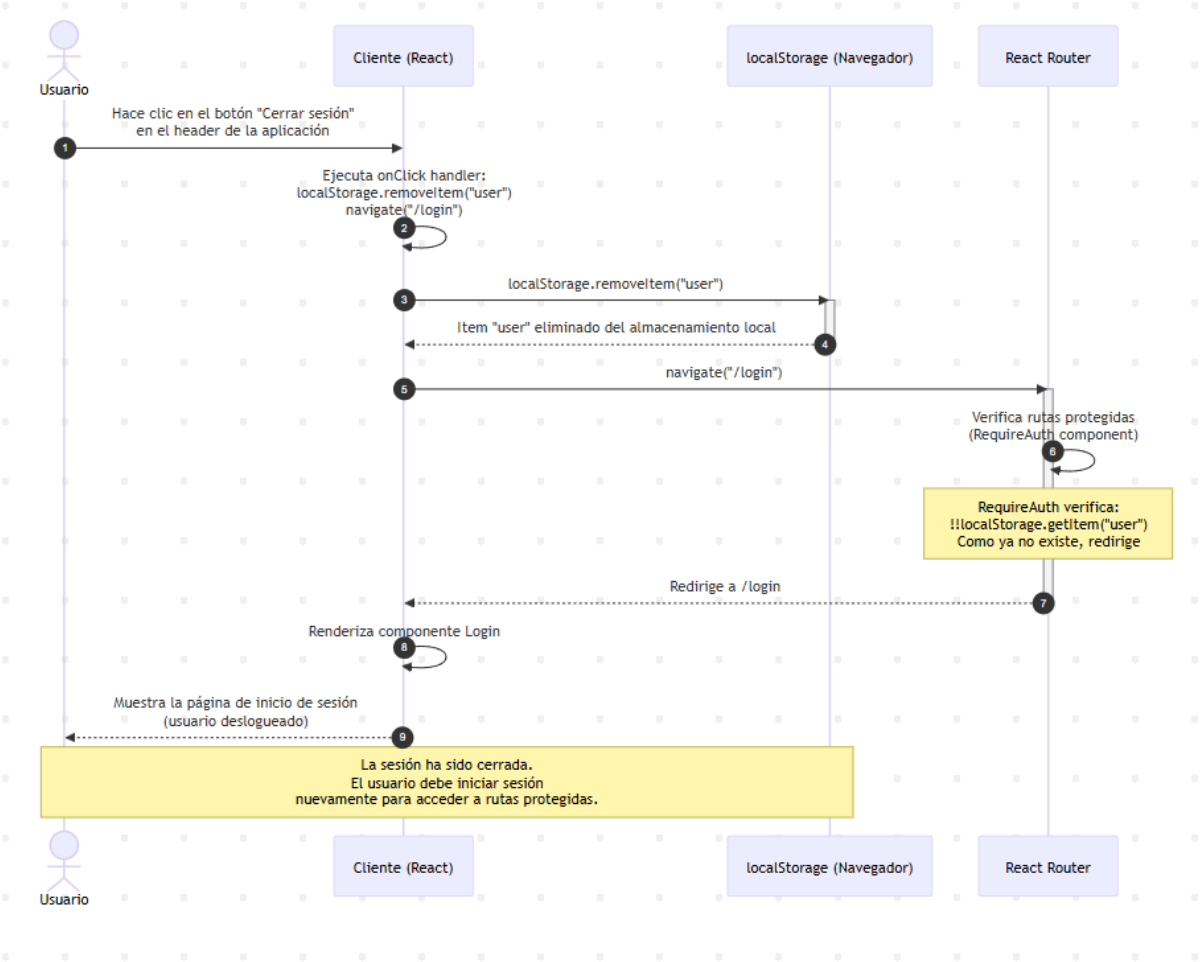
NOMBRE DEL CASO	Establecer nueva contraseña
DESCRIPCION	Un usuario que ha olvidado su contraseña introduce una nueva
ACTORES	Usuario invitado
PRECONDICIONES	El usuario debe haber accedido a la vista mediante un enlace válido que contenga un token de recuperación
SECUENCIA NORMAL	1. El usuario introduce su nueva contraseña y la confirmación. 2. El usuario pulsa el botón "Cambiar Contraseña". 3. El sistema verifica localmente que ambas contraseñas coinciden. 4. El sistema activa el estado de carga y envía el *token* y la contraseña al servidor. 5. El servidor valida el *token* y actualiza la contraseña en la base de datos. 6. El servidor devuelve una respuesta de éxito. 7. El sistema desactiva la carga, muestra un mensaje de éxito y, tras 2 segundos, redirige al Login.
POSTCONDICIONES	La contraseña del usuario se actualiza en la BBDD
EXCEPCIONES	- **Contraseñas distintas:** El botón se deshabilita y se muestra *"Las contraseñas no coinciden"*. No hay petición al servidor. - **Token inválido/caducado o error de backend:** El servidor rechaza la petición. Se muestra el mensaje del backend o *"No se pudo cambiar la contraseña"*. El estado de carga se desactiva. - **Caída de red:** Falla la conexión (catch). Se muestra el mensaje *"No se pudo conectar con el servidor..."* y el estado de carga se desactiva en el bloque finally.

- **Tabla 3.** Tabla de especificación de requisitos referente a la recuperación de contraseña.

- **Diagramas 7 y 8 y Tabla 4.** Cierre de sesión.



- **Diagrama 7.** Diagrama de Actividad referente al Cierre de Sesión.



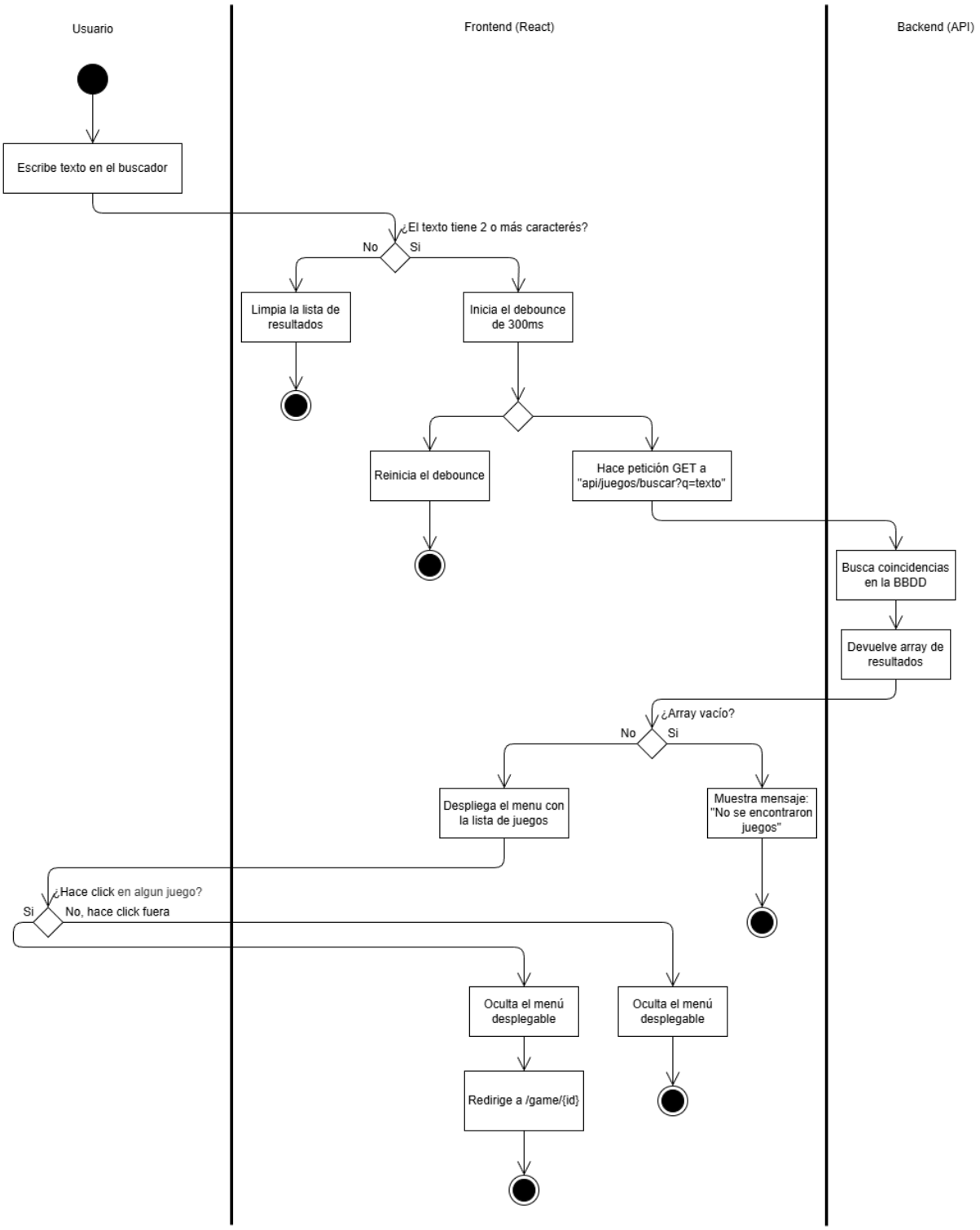
- **Diagrama 8.** Diagrama de Secuencia referente al Cierre de Sesión.

NOMBRE DEL CASO	Cerrar Sesión
DESCRIPCION	El usuario finaliza su sesión pulsando un botón
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe tener activa la sesión y estar en una pantalla que contenga el encabezado
SECUENCIA NORMAL	1. El usuario hace clic en el icono de "Cerrar sesión". 2. 2. El sistema elimina los datos de sesión del almacenamiento local del navegador. 3. 3. El sistema redirige al usuario a la pantalla de Inicio de Sesión.
POSTCONDICIONES	La sesión queda cerrada y el usuario no puede realizar ciertas acciones en el sistema.
EXCEPCIONES	Ninguna

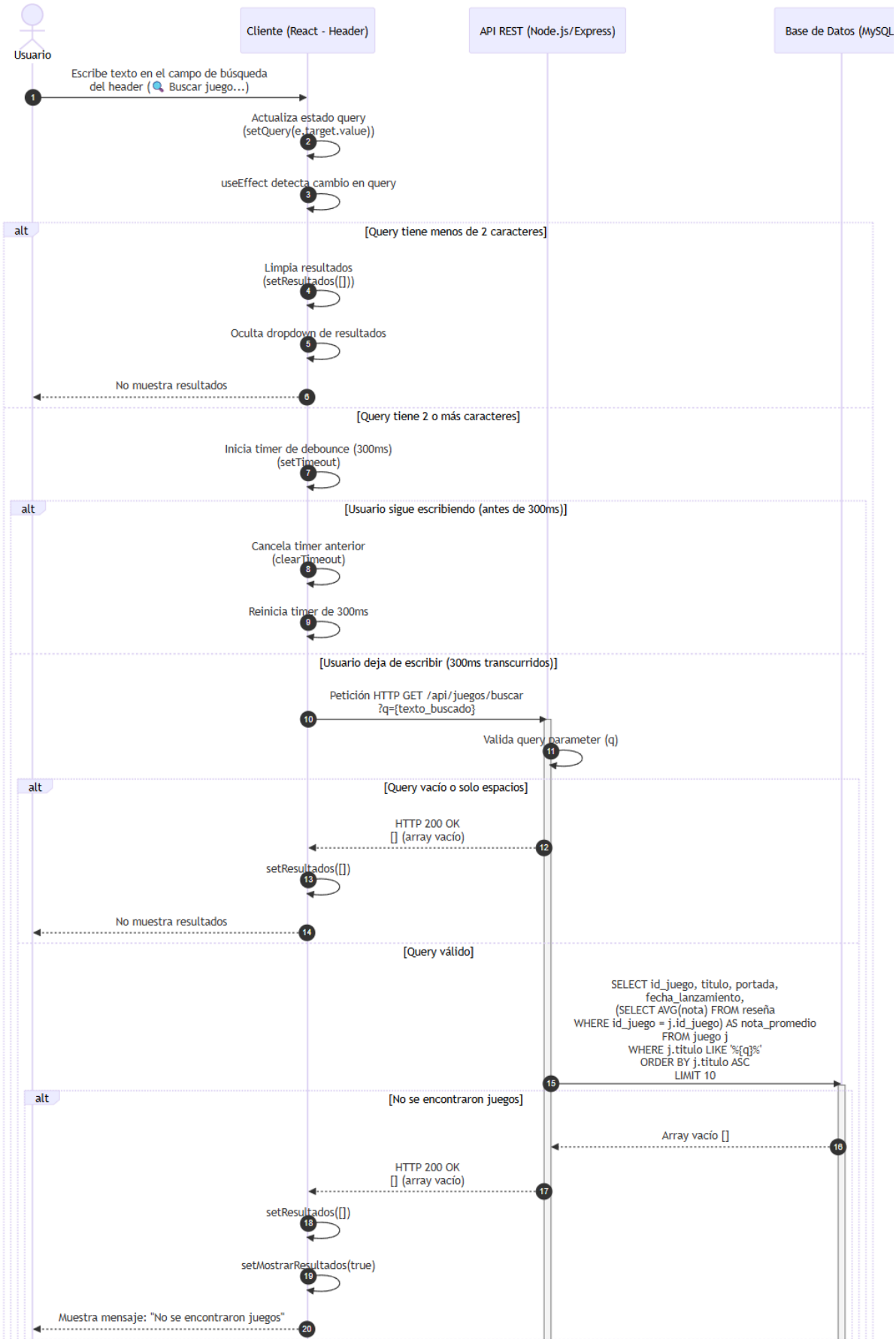
- **Tabla 4.** Tabla de especificación de requisitos referente al Cierre de Sesión.

Bloque B: Gestión de la Colección

- **Diagramas 9 y 10 y Tabla 5.** Buscar Juego.



- **Diagrama 9.** Diagrama de Actividad referente a Buscar Juego.

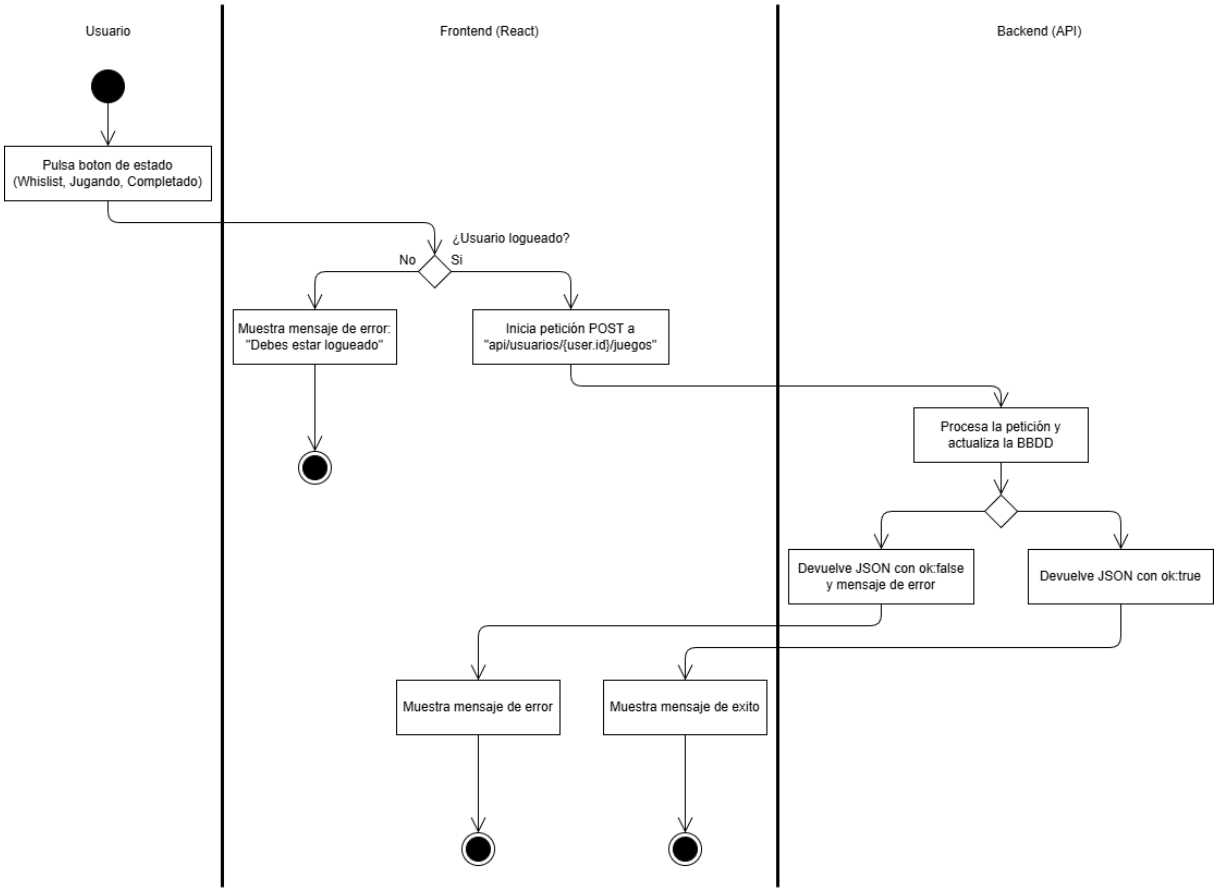

Usuario
Cliente (React - Header)
API REST (Node.js/Express)
Base de Datos (MySQL
Escribe texto en el campo de búsqueda del header (🔍 Buscar juego...)
Actualiza estado query (setQuery(e.target.value))
useEffect detecta cambio en query
alt
[Query tiene menos de 2 caracteres]
Limpia resultados (setResultados([]))
Oculta dropdown de resultados
No muestra resultados
[Query tiene 2 o más caracteres]
Inicia timer de debounce (300ms) (setTimeout)
alt
[Usuario sigue escribiendo (antes de 300ms)]
Cancela timer anterior (clearTimeout)
Reinicia timer de 300ms
[Usuario deja de escribir (300ms transcurridos)]
Petición HTTP GET /api/juegos/buscar ?q={texto_buscado}
Valida query parameter (q)
alt
[Query vacío o solo espacios]
HTTP 200 OK [] (array vacío)
setResultados([])
No muestra resultados
[Query válido]
SELECT id_juego, titulo, portada, fecha_lanzamiento, (SELECT AVG(nota) FROM reseña WHERE id_juego = j.id_juego) AS nota_promedio FROM juego j WHERE j.titulo LIKE '%{q}%' ORDER BY j.titulo ASC LIMIT 10
alt
[No se encontraron juegos]
Array vacío []
HTTP 200 OK [] (array vacío)
setResultados([])
setMostrarResultados(true)
Muestra mensaje: "No se encontraron juegos"

- **Diagrama 10.** Diagrama de Secuencia referente a Buscar Juego.

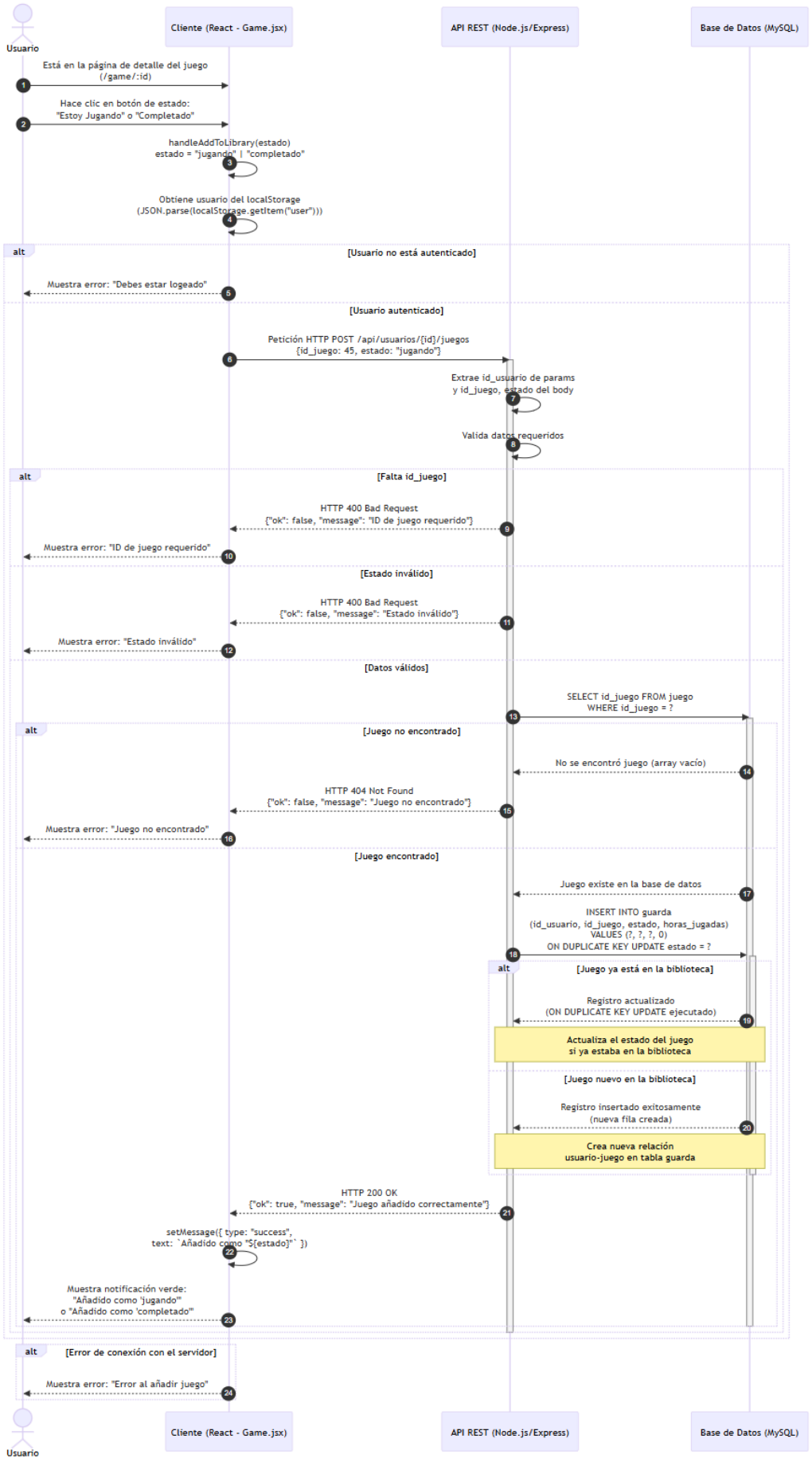
NOMBRE DEL CASO	Buscar Juego
DESCRIPCION	El usuario busca un juego en la barra de búsqueda mientras que el sistema muestra los posibles resultados en tiempo real.
ACTORES	Usuario registrado
PRECONDICIONES	El usuario debe estar en una pantalla que contenga el encabezado
SECUENCIA NORMAL	1. El usuario teclea el nombre de un juego (mínimo 2 caracteres). 2. El sistema espera 300ms sin nueva entrada. 3. El sistema realiza una petición al servidor con el texto de búsqueda. 4. El servidor devuelve una lista de coincidencias. 5. El sistema despliega una lista con las miniaturas, títulos y valoraciones. 6. El usuario hace clic en un juego de la lista. 7. El sistema limpia el buscador y redirige a la ficha del juego.
POSTCONDICIONES	El usuario se encuentra en la página con los detalles del juego seleccionado
EXCEPCIONES	- **Menos de 2 caracteres:** El sistema vacía la lista localmente y no hace petición al servidor. - **Sin resultados:** El servidor devuelve un array vacío. El sistema muestra *"No se encontraron juegos para [texto]"*. - **Clic fuera del buscador:** El sistema oculta el menú desplegable sin borrar el texto. - **Error de red/API:** El error se captura silenciosamente en consola y no se muestran resultados.

- **Tabla 5.** Tabla de especificación de requisitos referente al Buscar Juego.

- **Diagramas 11 y 12 y Tabla 6.** Añadir juego a biblioteca.



- **Diagrama 11.** Diagrama de Actividad referente a Añadir juego a biblioteca.

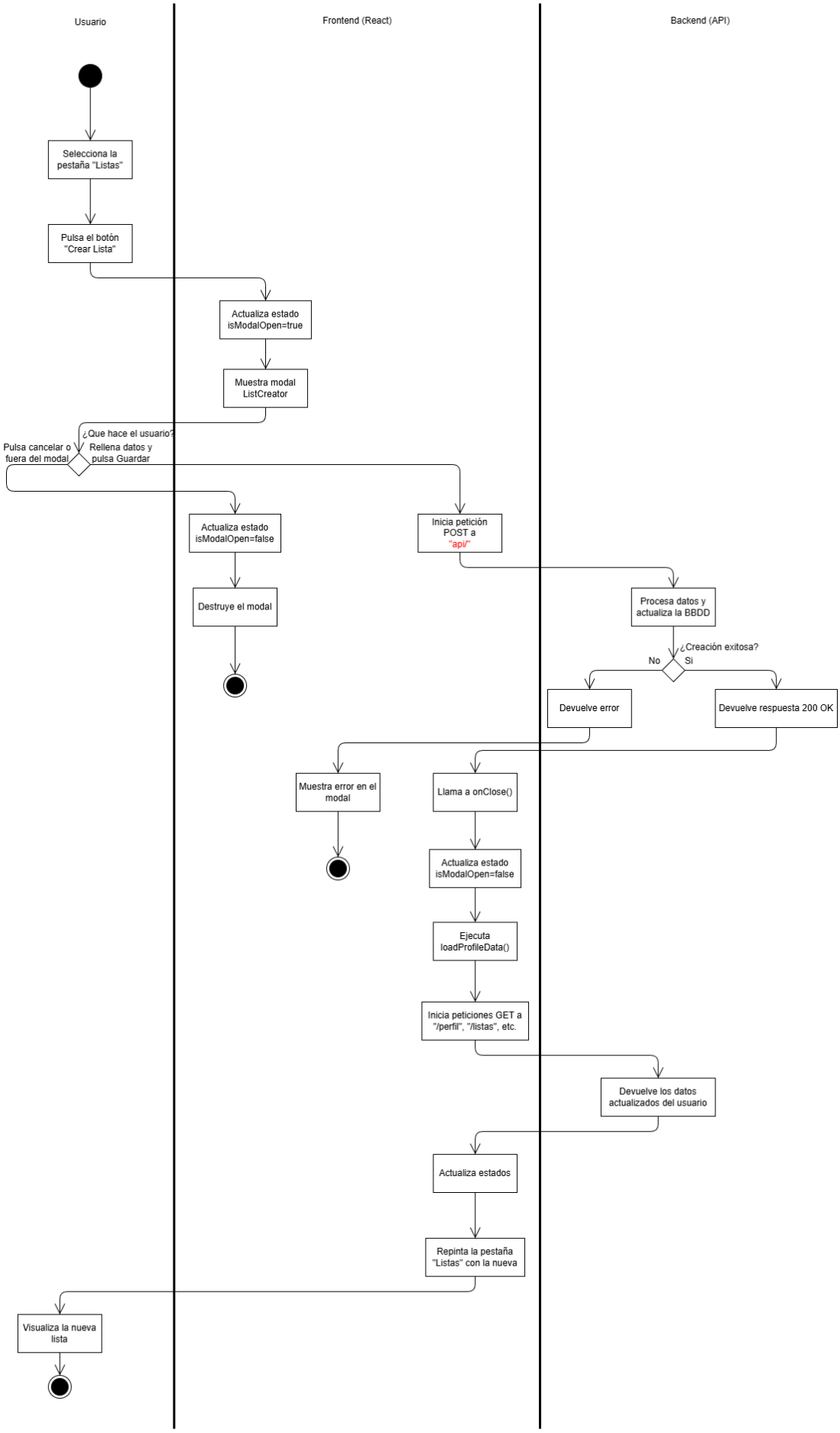

Usuario
Cliente (React - Game.jsx)
API REST (Node.js/Express)
Base de Datos (MySQL)
Está en la página de detalle del juego
(/game/:id)
Hace clic en botón de estado:
"Estoy Jugando" o "Completado"
handleAddToLibrary(estado)
estado = "jugando" | "completado"
Obtiene usuario del localStorage
(JSON.parse(localStorage.getItem("user")))
alt
[Usuario no está autenticado]
Muestra error: "Debes estar logeado"
[Usuario autenticado]
Petición HTTP POST /api/usuarios/{id}/juegos
{id_juego: 45, estado: "jugando"}
Extrae id_usuario de params
y id_juego, estado del body
Valida datos requeridos
alt
[Falta id_juego]
HTTP 400 Bad Request
{"ok": false, "message": "ID de juego requerido"}
Muestra error: "ID de juego requerido"
[Estado inválido]
HTTP 400 Bad Request
{"ok": false, "message": "Estado inválido"}
Muestra error: "Estado inválido"
[Datos válidos]
SELECT id_juego FROM juego
WHERE id_juego = ?
alt
[Juego no encontrado]
No se encontró juego (array vacío)
HTTP 404 Not Found
{"ok": false, "message": "Juego no encontrado"}
Muestra error: "Juego no encontrado"
[Juego encontrado]
Juego existe en la base de datos
INSERT INTO guarda
(id_usuario, id_juego, estado, horas_jugadas)
VALUES (?, ?, ?, 0)
ON DUPLICATE KEY UPDATE estado = ?
alt
[Juego ya está en la biblioteca]
Registro actualizado
(ON DUPLICATE KEY UPDATE ejecutado)
Actualiza el estado del juego
si ya estaba en la biblioteca
[Juego nuevo en la biblioteca]
Registro insertado exitosamente
(nueva fila creada)
Crea nueva relación
usuario-juego en tabla guarda
HTTP 200 OK
{"ok": true, "message": "Juego añadido correctamente"}
setMessage({ type: "success",
text: `Añadido como "${estado}"` })
Muestra notificación verde:
"Añadido como 'jugando'"
o "Añadido como 'completado'"
alt
[Error de conexión con el servidor]
Muestra error: "Error al añadir juego"
Usuario
Cliente (React - Game.jsx)
API REST (Node.js/Express)
Base de Datos (MySQL)

- **Diagrama 12.** Diagrama de Secuencia referente a Añadir juego a biblioteca.

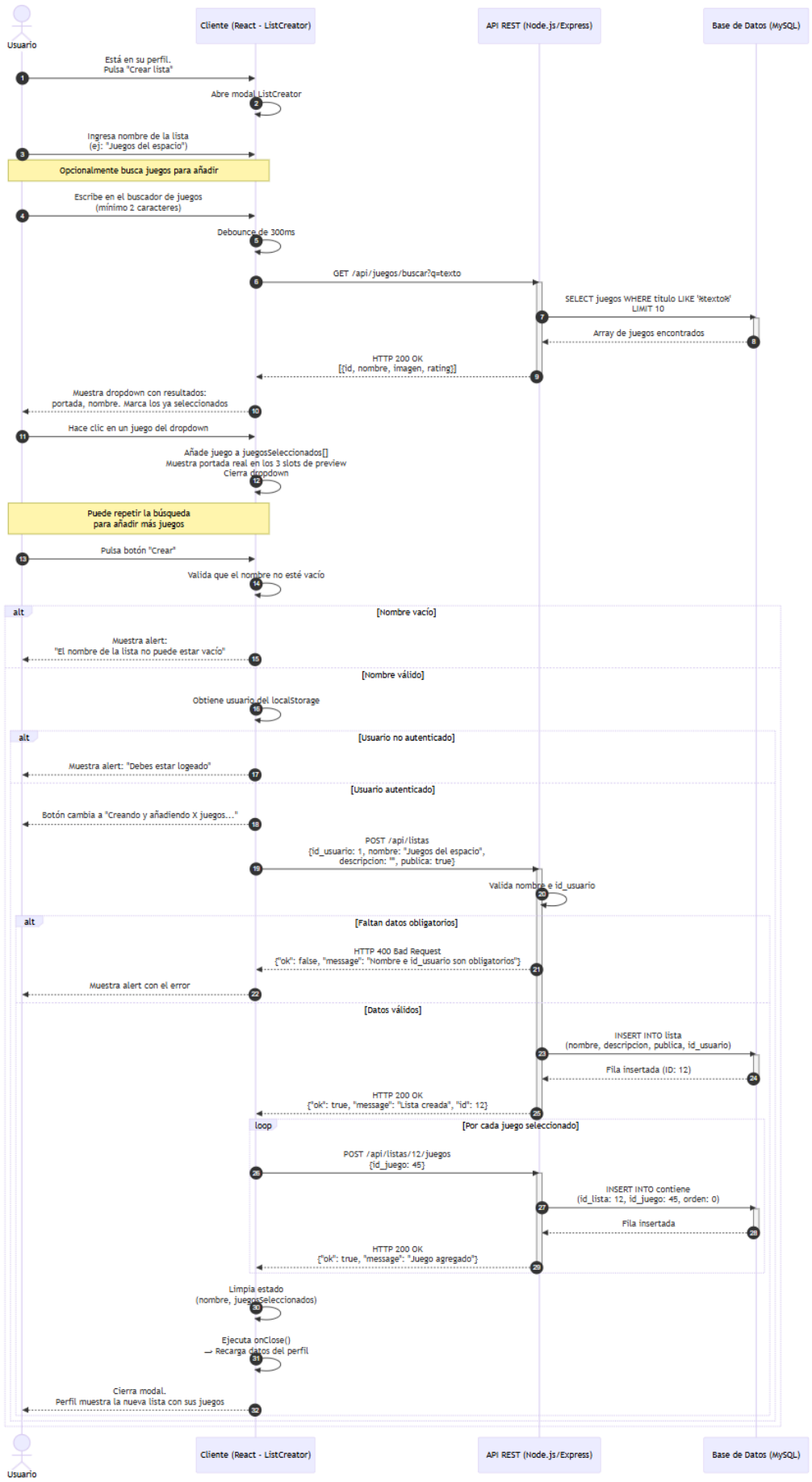
NOMBRE DEL CASO	Añadir juego a biblioteca de usuario
DESCRIPCION	El usuario categoriza un juego en su biblioteca personal pulsando uno de los botones de estado (Wishlist, Jugando, Completado).
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe estar logueado
SECUENCIA NORMAL	1. El usuario hace clic en el botón del estado deseado. 2. El sistema verifica que existe una sesión activa. 3. El sistema realiza una petición POST a /api/usuarios/{id}/juegos con el ID del juego y el estado. 4. El servidor actualiza el registro en la BBDD. 5. El servidor responde afirmativamente. 6. El sistema muestra un mensaje de éxito ("Añadido como...").
POSTCONDICIONES	El juego queda registrado en la biblioteca del usuario con su nuevo estado.
EXCEPCIONES	- **Usuario no logueado:** El sistema detiene el flujo y muestra el error *"Debes estar logeado"*. No se realiza petición al servidor. - **Error del servidor / Red:** El servidor devuelve error o falla la conexión. El sistema captura la excepción y muestra el error correspondiente.

- **Tabla 6.** Tabla de especificación de requisitos referente a Añadir juego a biblioteca.

- **Diagramas 13 y 14 y Tabla 7.** Crear Lista.


Usuario
Frontend (React)
Backend (API)
Selecciona la pestaña "Listas"
Pulsa el botón "Crear Lista"
Actualiza estado isModalOpen=true
Muestra modal ListCreator
¿Que hace el usuario?
Pulsa cancelar o fuera del modal
Rellena datos y pulsa Guardar
Actualiza estado isModalOpen=false
Destruye el modal
Inicia petición POST a "api/"
Procesa datos y actualiza la BBDD
¿Creación exitosa?
No
Si
Devuelve error
Devuelve respuesta 200 OK
Muestra error en el modal
Llama a onClose()
Actualiza estado isModalOpen=false
Ejecuta loadProfileData()
Inicia peticiones GET a "/perfil", "/listas", etc.
Devuelve los datos actualizados del usuario
Actualiza estados
Repinta la pestaña "Listas" con la nueva
Visualiza la nueva lista

- **Diagrama 13.** Diagrama de Actividad referente a Crear Lista.

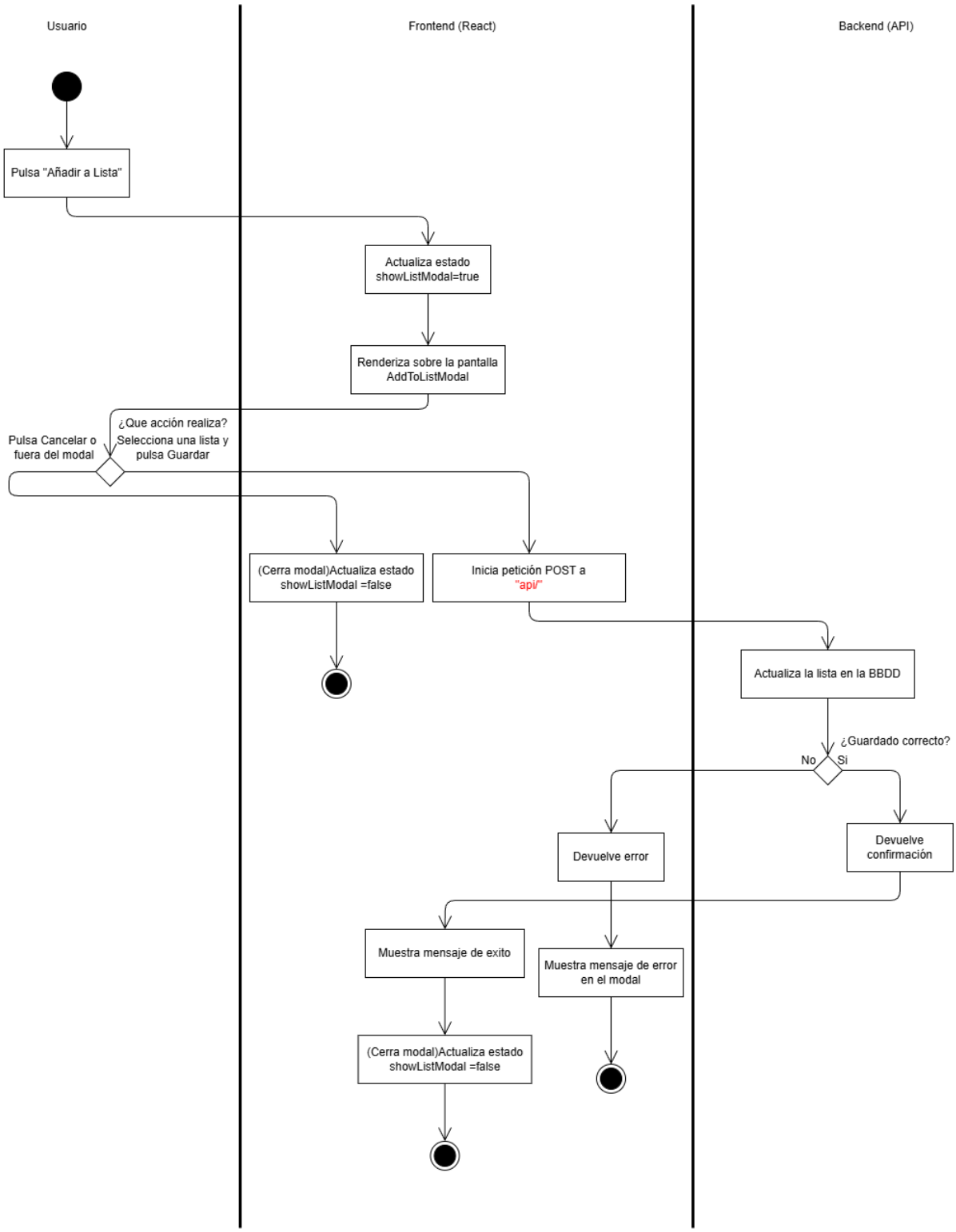

Usuario
Cliente (React - ListCreator)
API REST (Node.js/Express)
Base de Datos (MySQL)
Está en su perfil.
Pulsa "Crear lista"
Abre modal ListCreator
Ingresa nombre de la lista
(ej: "Juegos del espacio")
Opcionalmente busca juegos para añadir
Escribe en el buscador de juegos
(mínimo 2 caracteres)
Debounce de 300ms
GET /api/juegos/buscar?q=texto
SELECT juegos WHERE titulo LIKE '%texto%'
LIMIT 10
Array de juegos encontrados
HTTP 200 OK
[{id, nombre, imagen, rating}]
Muestra dropdown con resultados:
portada, nombre. Marca los ya seleccionados
Hace clic en un juego del dropdown
Añade juego a juegosSeleccionados[]
Muestra portada real en los 3 slots de preview
Cierra dropdown
Puede repetir la búsqueda
para añadir más juegos
Pulsa botón "Crear"
Valida que el nombre no esté vacío
alt
[Nombre vacío]
Muestra alert:
"El nombre de la lista no puede estar vacío"
[Nombre válido]
Obtiene usuario del localStorage
alt
[Usuario no autenticado]
Muestra alert: "Debes estar logeado"
[Usuario autenticado]
Botón cambia a "Creando y añadiendo X juegos..."
POST /api/listas
{id_usuario: 1, nombre: "Juegos del espacio",
descripcion: "", publica: true}
Valida nombre e id_usuario
alt
[Faltan datos obligatorios]
HTTP 400 Bad Request
{"ok": false, "message": "Nombre e id_usuario son obligatorios"}
Muestra alert con el error
[Datos válidos]
INSERT INTO lista
(nombre, descripcion, publica, id_usuario)
Fila insertada (ID: 12)
HTTP 200 OK
{"ok": true, "message": "Lista creada", "id": 12}
loop
[Por cada juego seleccionado]
POST /api/listas/12/juegos
{id_juego: 45}
INSERT INTO contiene
(id_lista: 12, id_juego: 45, orden: 0)
Fila insertada
HTTP 200 OK
{"ok": true, "message": "Juego agregado"}
Limpia estado
(nombre, juegosSeleccionados)
Ejecuta onClose()
→ Recarga datos del perfil
Cierra modal.
Perfil muestra la nueva lista con sus juegos

- **Diagrama 14.** Diagrama de Secuencia referente a Crear Lista.

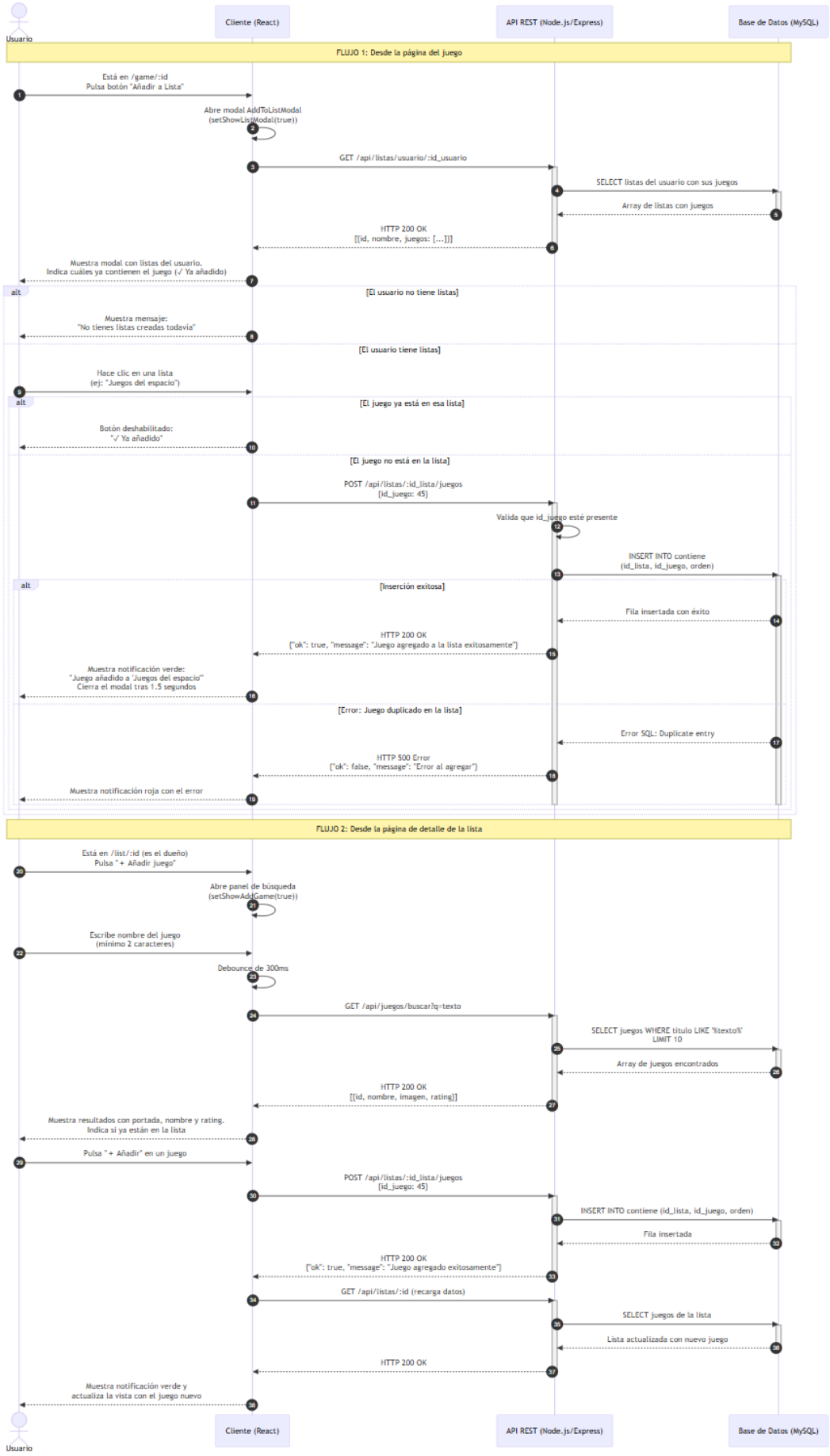
NOMBRE DEL CASO	Crear Lista
DESCRIPCION	Un usuario autenticado crea una nueva lista de videojuegos desde la pestaña "Listas" de su propio perfil.
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe estar logueado y estar visualizando su propio perfil
SECUENCIA NORMAL	1. El usuario navega a la pestaña "Listas" y pulsa "Crear lista". 2. El sistema despliega el modal. 3. El usuario introduce los datos de la lista y pulsa "Guardar". 4. El sistema envía una petición de creación al servidor. 5. El servidor registra la nueva lista en la base de datos y responde con éxito. 6. 6. El sistema cierra el modal. 7. 7. El sistema ejecuta una recarga automática de los datos del perfil (loadProfileData). 8. 8. El servidor devuelve las listas actualizadas y el sistema las renderiza en pantalla.
POSTCONDICIONES	La lista se ha creado en la BBDD, vinculada al usuario y aparece en la UI.
EXCEPCIONES	- **Cierre voluntario:** El usuario pulsa "Cancelar" o cierra el modal. El estado isModalOpen pasa a false y no se hace petición a la API. - **Error al guardar:** Si la API devuelve un error, el componente muestra el error y el modal no se cierra para permitir correcciones.

- **Tabla 7.** Tabla de especificación de requisitos referente a Crear Lista.

- **Diagramas 15 y 16 y Tabla 8.** Añadir juego a lista.



- **Diagrama 15.** Diagrama de Actividad referente a Añadir juego a lista.


Usuario
Cliente (React)
API REST (Node.js/Express)
Base de Datos (MySQL)
FLUJO 1: Desde la página del juego
Está en /game/:id
Pulsa botón "Añadir a Lista"
Abre modal AddToListModal
(setShowListModal(true))
GET /api/listas/usuario/:id_usuario
SELECT listas del usuario con sus juegos
Array de listas con juegos
HTTP 200 OK
[{id, nombre, juegos: [...]}]
Muestra modal con listas del usuario.
Indica cuáles ya contienen el juego (✓ Ya añadido)
alt
[El usuario no tiene listas]
Muestra mensaje:
"No tienes listas creadas todavía"
[El usuario tiene listas]
Hace clic en una lista
(ej: "Juegos del espacio")
alt
[El juego ya está en esa lista]
Botón deshabilitado:
"✓ Ya añadido"
[El juego no está en la lista]
POST /api/listas/:id_lista/juegos
{id_juego: 45}
Valida que id_juego esté presente
INSERT INTO contiene
(id_lista, id_juego, orden)
alt
[Inserción exitosa]
Fila insertada con éxito
HTTP 200 OK
{"ok": true, "message": "Juego agregado a la lista exitosamente"}
Muestra notificación verde:
"Juego añadido a "Juegos del espacio"
Cierra el modal tras 1.5 segundos
[Error: Juego duplicado en la lista]
Error SQL: Duplicate entry
HTTP 500 Error
{"ok": false, "message": "Error al agregar"}
Muestra notificación roja con el error
FLUJO 2: Desde la página de detalle de la lista
Está en /list/:id (es el dueño)
Pulsa "+ Añadir juego"
Abre panel de búsqueda
(setShowAddGame(true))
Escribe nombre del juego
(mínimo 2 caracteres)
Debounce de 300ms
GET /api/juegos/buscar?q=texto
SELECT juegos WHERE titulo LIKE '%texto%'
LIMIT 10
Array de juegos encontrados
HTTP 200 OK
[{id, nombre, imagen, rating}]
Muestra resultados con portada, nombre y rating.
Indica si ya están en la lista
Pulsa "+ Añadir" en un juego
POST /api/listas/:id_lista/juegos
{id_juego: 45}
INSERT INTO contiene (id_lista, id_juego, orden)
Fila insertada
HTTP 200 OK
{"ok": true, "message": "Juego agregado exitosamente"}
GET /api/listas/:id (recarga datos)
SELECT juegos de la lista
Lista actualizada con nuevo juego
HTTP 200 OK
Muestra notificación verde y
actualiza la vista con el juego nuevo
Usuario
Cliente (React)
API REST (Node.js/Express)
Base de Datos (MySQL)

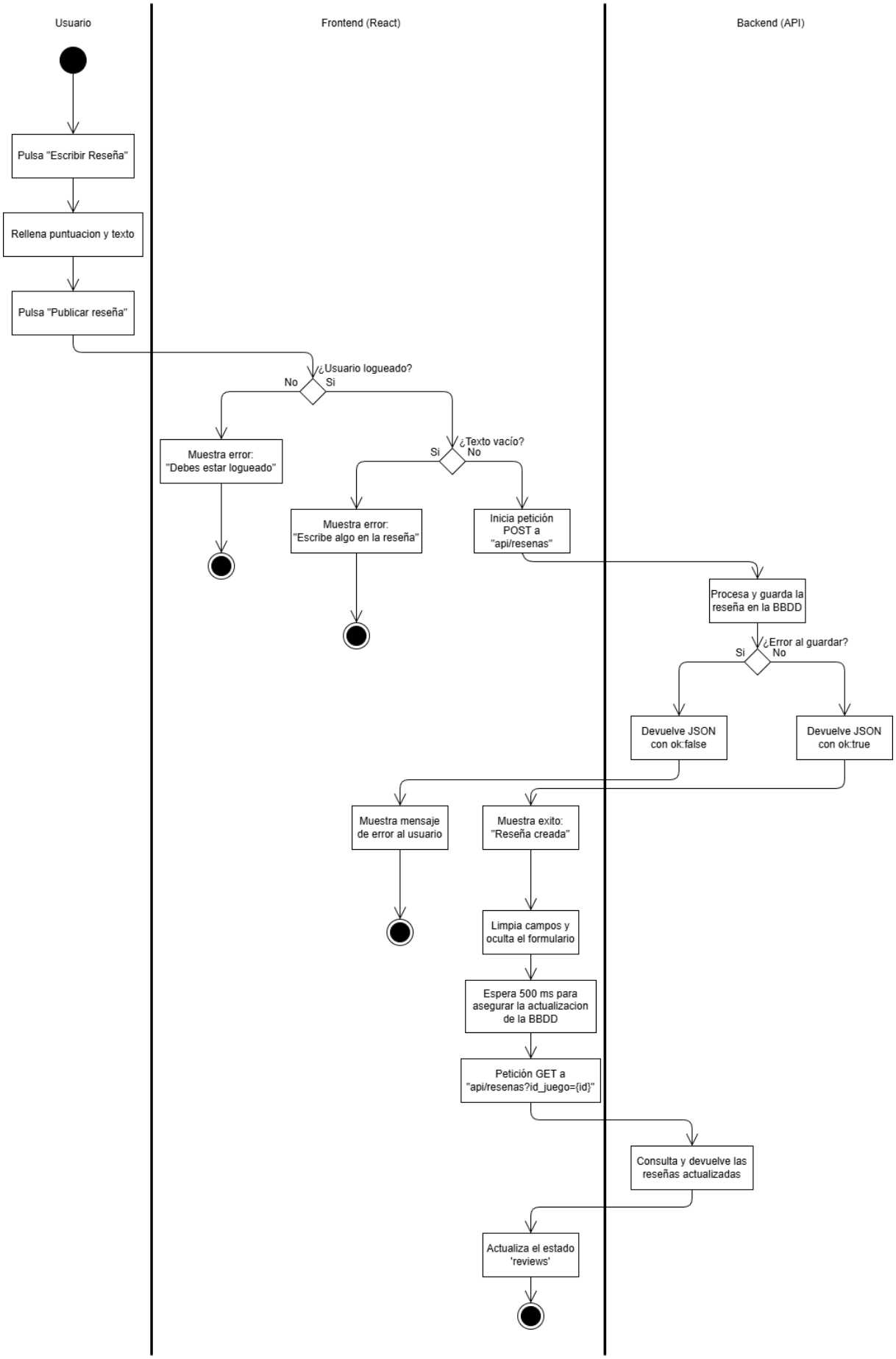
- **Diagrama 16.** Diagrama de Secuencia referente a Añadir juego a lista.

NOMBRE DEL CASO	Añadir juego a lista
DESCRIPCION	El usuario abre el modal de listas para añadir el juego actual a una de sus listas personalizadas creadas previamente.
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe estar logueado
SECUENCIA NORMAL	1. El usuario pulsa "Añadir a Lista". 2. El sistema renderiza el modal superpuesto. 3. El usuario selecciona una lista de las disponibles en el modal. 4. El sistema envía una petición al servidor para vincular el juego con la lista elegida. 5. El servidor confirma la operación. 6. El sistema cierra el modal y muestra un mensaje de éxito.
POSTCONDICIONES	El juego queda vinculado a la lista seleccionada por el usuario.
EXCEPCIONES	- **Cierre voluntario:** El usuario pulsa fuera del modal o en el botón de cerrar. El sistema desmonta el modal sin hacer cambios. - **Error al guardar:** Si la API falla, el modal muestra un error en lugar de cerrarse.

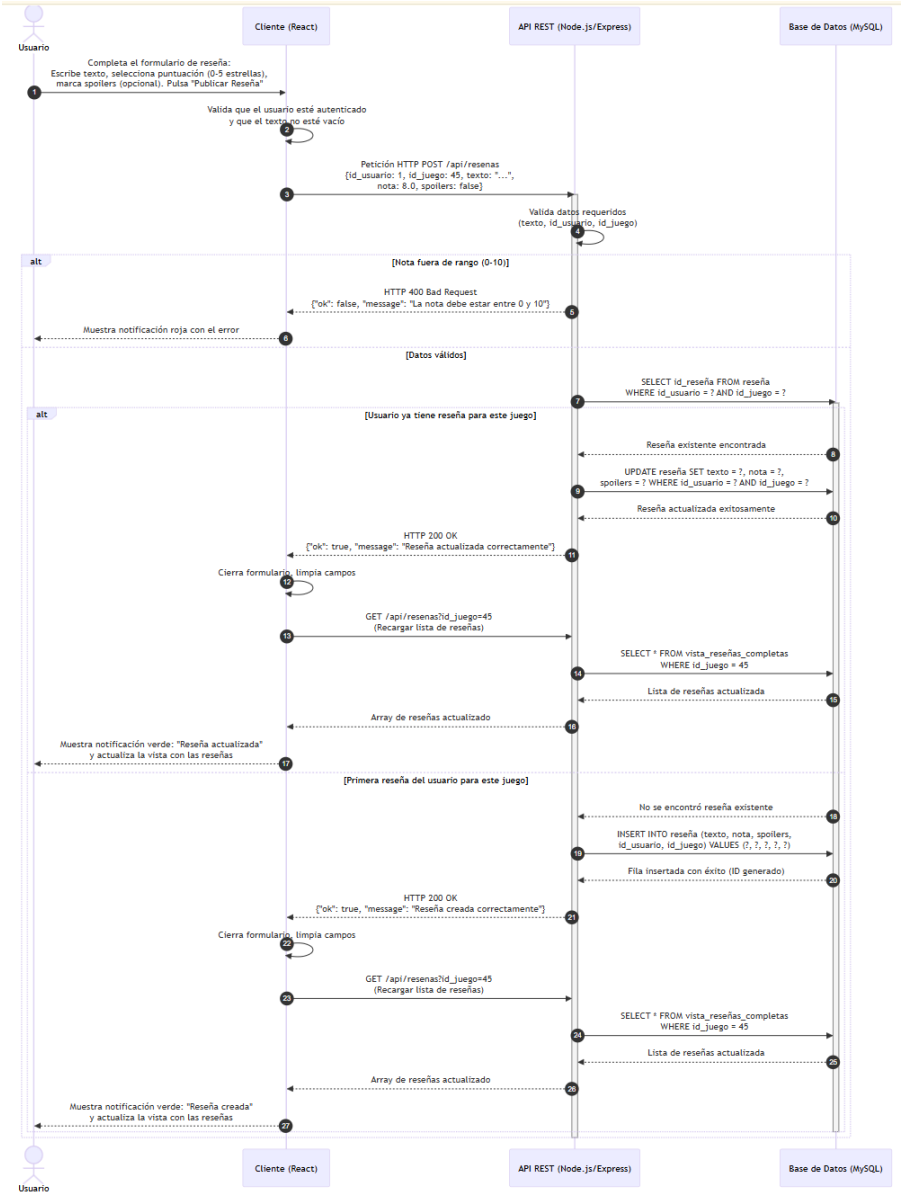
- **Tabla 8.** Tabla de especificación de requisitos referente a Añadir juego a lista.

Bloque C: Interacción Social

- **Diagramas 15 y 16 y Tabla 8.** Publicar review.


Usuario
Frontend (React)
Backend (API)
Pulsa "Escribir Reseña"
Rellena puntuacion y texto
Pulsa "Publicar reseña"
¿Usuario logueado?
No
Si
Muestra error: "Debes estar logueado"
¿Texto vacío?
Si
No
Muestra error: "Escribe algo en la reseña"
Inicia petición POST a "api/resenas"
Procesa y guarda la reseña en la BBDD
¿Error al guardar?
Si
No
Devuelve JSON con ok:false
Devuelve JSON con ok:true
Muestra mensaje de error al usuario
Muestra exito: "Reseña creada"
Limpia campos y oculta el formulario
Espera 500 ms para asegurar la actualizacion de la BBDD
Petición GET a "api/resenas?id_juego={id}"
Consulta y devuelve las reseñas actualizadas
Actualiza el estado 'reviews'

- **Diagrama 15.** Diagrama de Actividad referente a Publicar review.



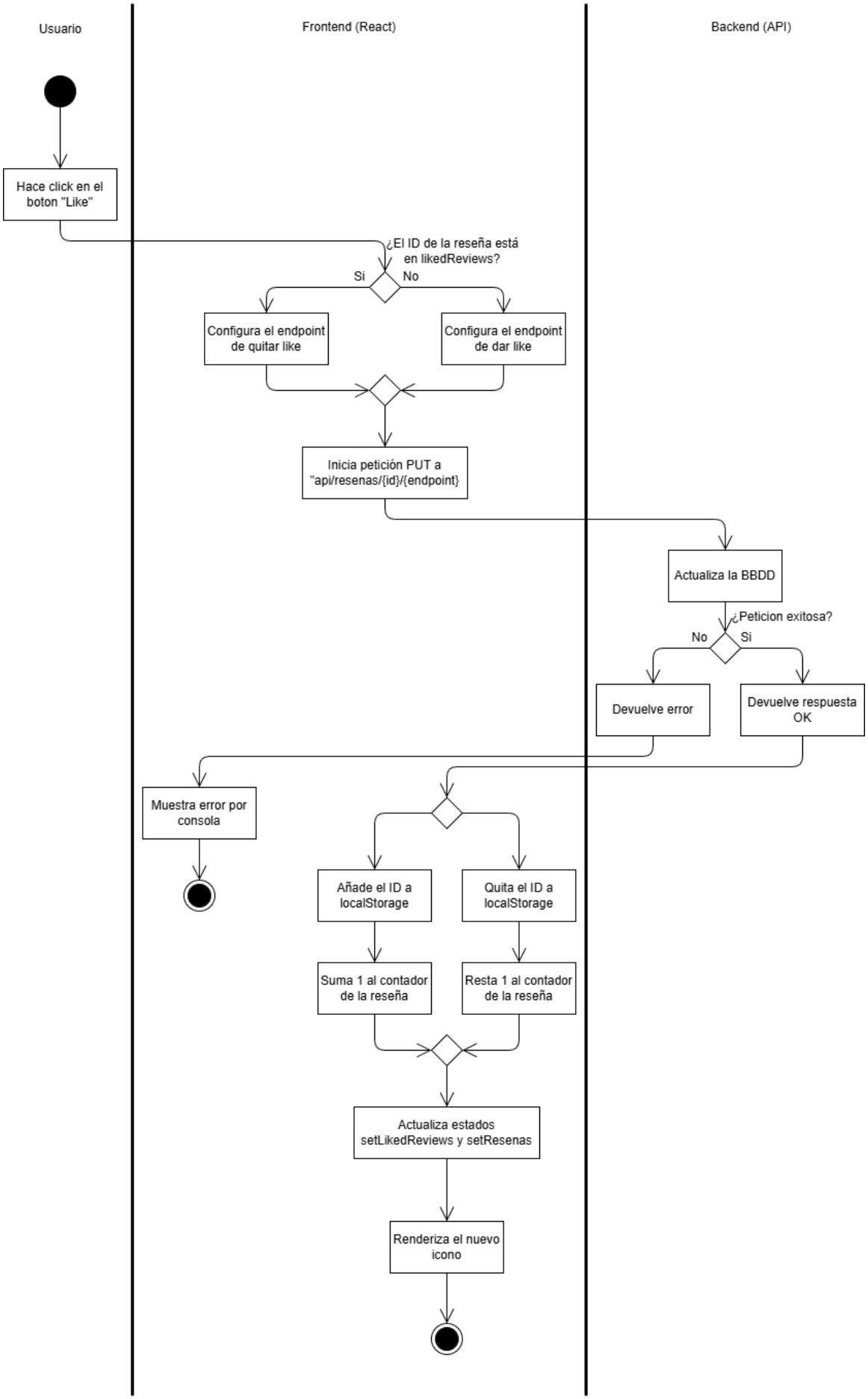
- **Diagrama 16.** Diagrama de Secuencia referente a Publicar review.

NOMBRE DEL CASO	Publicar review de un juego
DESCRIPCION	Un usuario registrado y con sesión activa valora y escribe una opinión sobre un juego en su página correspondiente
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe haber iniciado sesión y encontrarse en la página de detalles de un juego
SECUENCIA NORMAL	1. El usuario pulsa "Escribir Reseña" para desplegar el formulario.

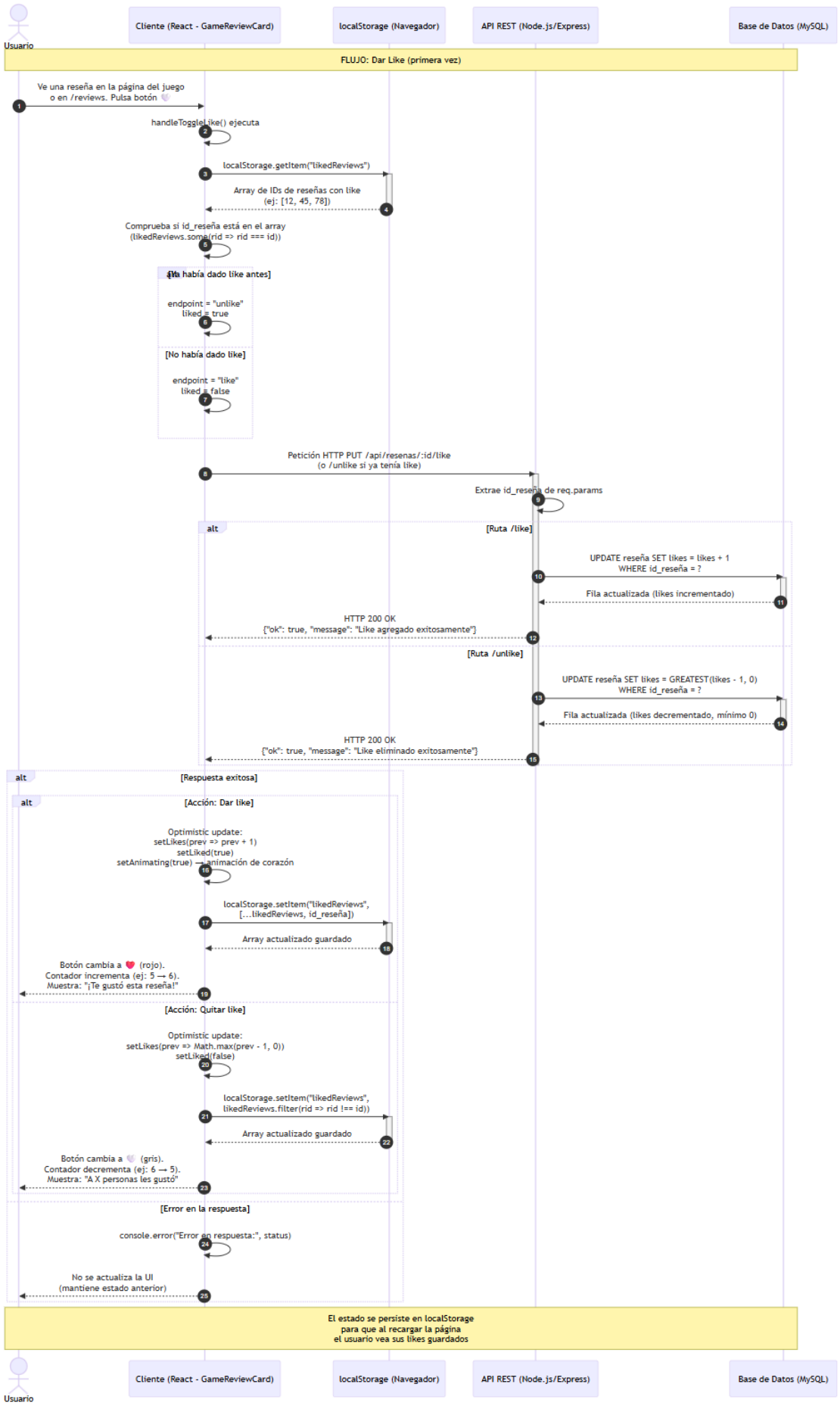
	2. Introduce una puntuación (estrellas) y el texto de la reseña. 3. Pulsa "Publicar Reseña". 4. El sistema valida que el usuario esté logueado y que el texto no esté vacío. 5. El sistema envía los datos al servidor. 6. El servidor guarda la reseña en la BBDD y responde con éxito. 7. El sistema muestra "Reseña creada", limpia y oculta el formulario. 8. Tras 500ms, el sistema vuelve a solicitar al servidor la lista de reseñas actualizada para mostrarla en pantalla.
POSTCONDICIONES	La reseña queda vinculada al juego y al usuario
EXCEPCIONES	- **Usuario no logueado:** Muestra el error *"Debes estar logeado"*. No se envía nada al servidor. - **Texto vacío:** Muestra el error *"Escribe algo en la reseña"*. No se envía nada al servidor. - **Error de servidor/red:** Muestra el error *"Error al crear reseña"* o el devuelto por la API.

- **Tabla 8.** Tabla de especificación de requisitos referente a Publicar review.

- **Diagramas 15 y 16 y Tabla 8.** Dar/Quitar like a review.


Usuario
Frontend (React)
Backend (API)
Hace click en el boton "Like"
¿El ID de la reseña está en likedReviews?
Si
No
Configura el endpoint de quitar like
Configura el endpoint de dar like
Inicia petición PUT a "api/resenas/{id}/{endpoint}
Actualiza la BBDD
¿Peticion exitosa?
No
Si
Devuelve error
Devuelve respuesta OK
Muestra error por consola
Añade el ID a localStorage
Quita el ID a localStorage
Suma 1 al contador de la reseña
Resta 1 al contador de la reseña
Actualiza estados setLikedReviews y setResenas
Renderiza el nuevo icono

- **Diagrama 15.** Diagrama de Actividad referente a Dar/Quitar like a review.

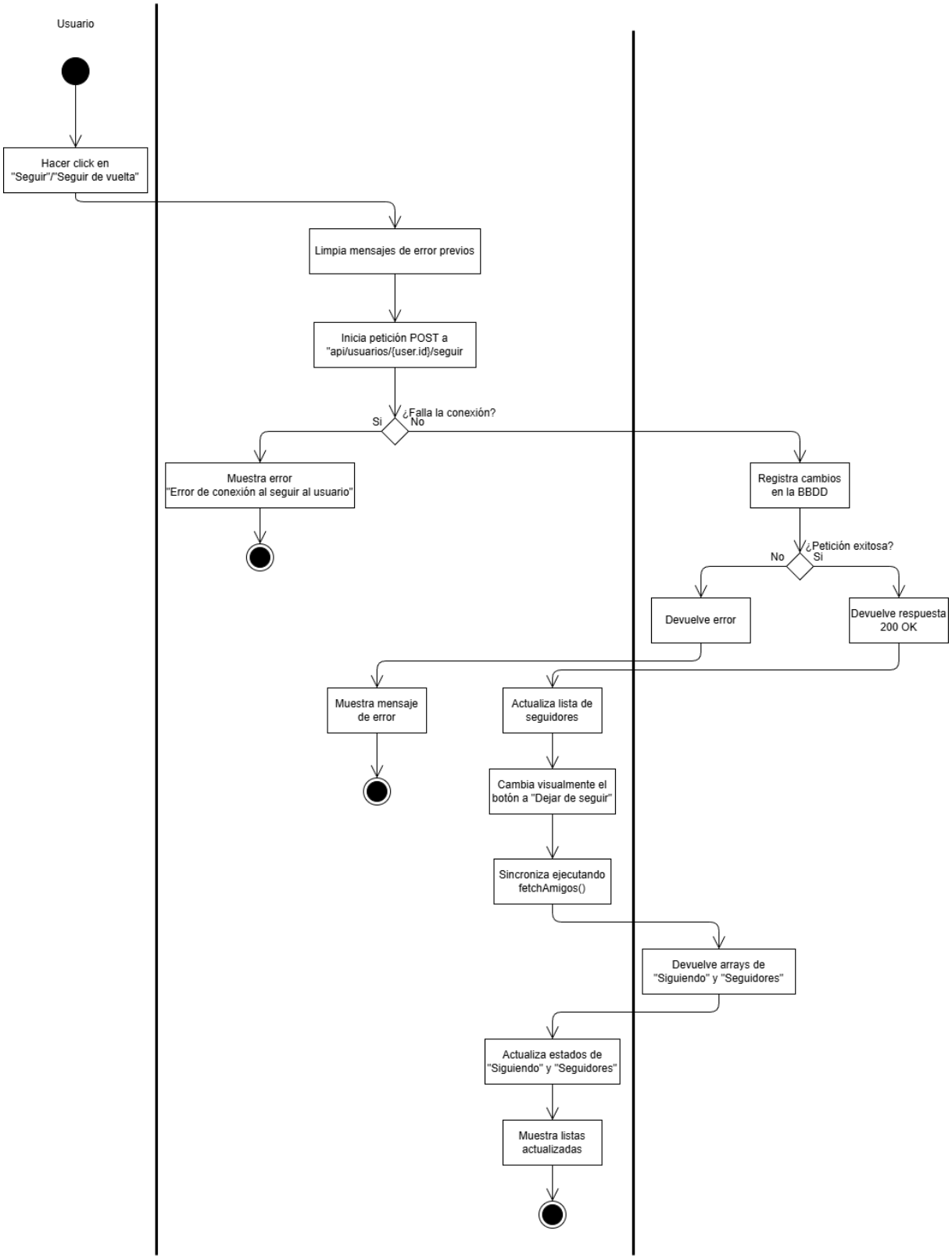

Usuario
Cliente (React - GameReviewCard)
localStorage (Navegador)
API REST (Node.js/Express)
Base de Datos (MySQL)
FLUJO: Dar Like (primera vez)
Ve una reseña en la página del juego o en /reviews. Pulsa botón
handleToggleLike() ejecuta
localStorage.getItem("likedReviews")
Array de IDs de reseñas con like (ej: [12, 45, 78])
Comprueba si id_reseña está en el array (likedReviews.some(rid => rid === id))
alt
[Ya había dado like antes]
endpoint = "unlike"
liked = true
[No había dado like]
endpoint = "like"
liked = false
Petición HTTP PUT /api/resenas/:id/like (o /unlike si ya tenía like)
Extrae id_reseña de req.params
alt
[Ruta /like]
UPDATE reseña SET likes = likes + 1 WHERE id_reseña = ?
Fila actualizada (likes incrementado)
HTTP 200 OK
{"ok": true, "message": "Like agregado exitosamente"}
[Ruta /unlike]
UPDATE reseña SET likes = GREATEST(likes - 1, 0) WHERE id_reseña = ?
Fila actualizada (likes decrementado, mínimo 0)
HTTP 200 OK
{"ok": true, "message": "Like eliminado exitosamente"}
alt
[Respuesta exitosa]
alt
[Acción: Dar like]
Optimistic update:
setLikes(prev => prev + 1)
setLiked(true)
setAnimating(true) → animación de corazón
localStorage.setItem("likedReviews", [...likedReviews, id_reseña])
Array actualizado guardado
Botón cambia a (rojo).
Contador incrementa (ej: 5 → 6).
Muestra: "¡Te gustó esta reseña!"
[Acción: Quitar like]
Optimistic update:
setLikes(prev => Math.max(prev - 1, 0))
setLiked(false)
localStorage.setItem("likedReviews", likedReviews.filter(rid => rid !== id))
Array actualizado guardado
Botón cambia a (gris).
Contador decrementa (ej: 6 → 5).
Muestra: "A X personas les gustó"
[Error en la respuesta]
console.error("Error en respuesta:", status)
No se actualiza la UI (mantiene estado anterior)
El estado se persiste en localStorage para que al recargar la página el usuario vea sus likes guardados
Usuario
Cliente (React - GameReviewCard)
localStorage (Navegador)
API REST (Node.js/Express)
Base de Datos (MySQL)

- **Diagrama 16.** Diagrama de Secuencia referente a Dar/Quitar like a review.

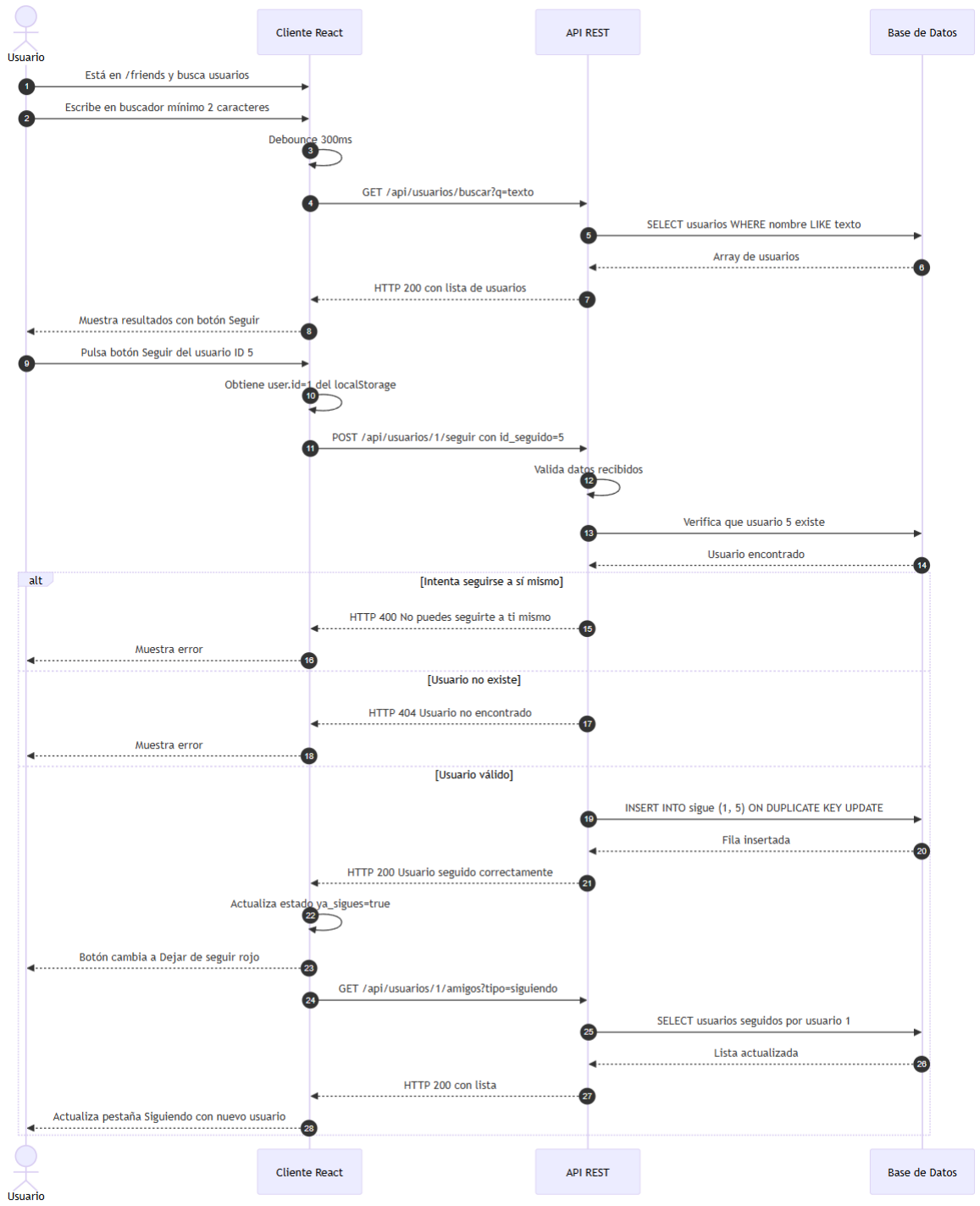
NOMBRE DEL CASO	Dar/quitar like a una review
DESCRIPCION	El usuario interactúa con el botón de "Me gusta" de una reseña para mostrar su aprecio o retirarlo.
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe visualizar la review.
SECUENCIA NORMAL	1. El usuario hace clic en el botón de Like de una reseña específica. 2. El sistema comprueba en el almacenamiento local si el usuario ya le había dado like previamente. 3. El sistema determina el *endpoint* correcto (/like o /unlike). 4. El sistema envía una petición PUT al servidor. 5. El servidor actualiza el contador en la base de datos y responde afirmativamente. 6. El sistema actualiza el registro local guardando o eliminando el ID de la reseña en localStorage. 7. El sistema actualiza el estado sumando o restando 1 al contador visualmente e invirtiendo el icono al instante.
POSTCONDICIONES	El contador de la reseña se actualiza en el servidor y la interfaz refleja el cambio sin recargar la página.
EXCEPCIONES	- **Error del servidor / Red:** Si la petición falla, el sistema captura el error por consola, pero la interfaz gráfica no se altera (se cancela el proceso).

- **Tabla 8.** Tabla de especificación de requisitos referente a Dar/Quitar like a review.

- **Diagramas 15 y 16 y Tabla 8.** Seguir a otro usuario.



- **Diagrama 15.** Diagrama de Actividad referente a Seguir a otro usuario.



- **Diagrama 16.** Diagrama de Secuencia referente a Seguir a otro usuario.

NOMBRE DEL CASO	Seguir a otro usuario registrado
DESCRIPCION	Un usuario autenticado busca a otro usuario en la plataforma y pulsa el botón para empezar a seguirlo.
ACTORES	Usuario Registrado
PRECONDICIONES	El usuario debe haber iniciado sesión y estar en la pantalla de "Amigos" (/friends).
SECUENCIA NORMAL	1. El usuario introduce el nombre en el buscador o visualiza su lista de seguidores.

	2. El usuario hace clic en el botón "Seguir" junto al nombre de otro usuario. 3. El sistema limpia errores previos y realiza una petición POST enviando el ID del usuario a seguir. 4. El servidor registra la relación (seguidor-seguido) en la base de datos y responde con éxito. 5. El sistema actualiza el estado local cambiando el estado del botón a "Dejar de seguir". 6. El sistema recarga la lista de "Siguiendo" y "Seguidores" actualizada desde el servidor.
POSTCONDICIONES	El usuario pasa a seguir al usuario seleccionado y las listas de la interfaz se actualizan.
EXCEPCIONES	- **Usuario no logueado:** Si no hay un user.id, el sistema muestra el error *"Debes iniciar sesión..."* al cargar la página. - **Error de servidor/red al seguir:** Si la petición POST falla, el sistema captura el error y muestra el mensaje.

- **Tabla 8.** Tabla de especificación de requisitos referente a Seguir a otro usuario.

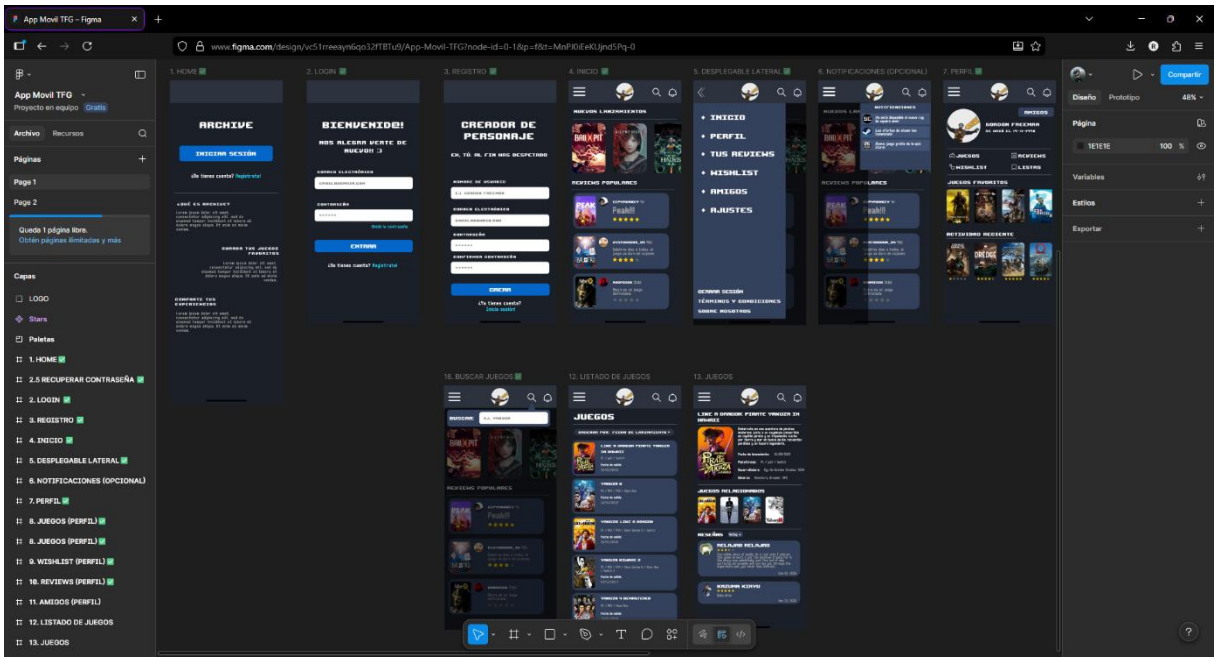
Anexo 3. Diseño Visual y Experiencia de Usuario (UX/UI)

- **Imagen 1.** Paleta de Colores.

Descripción: Definición de la identidad visual diseñada para resaltar elementos gráficos como carátulas y reducir la carga visual.

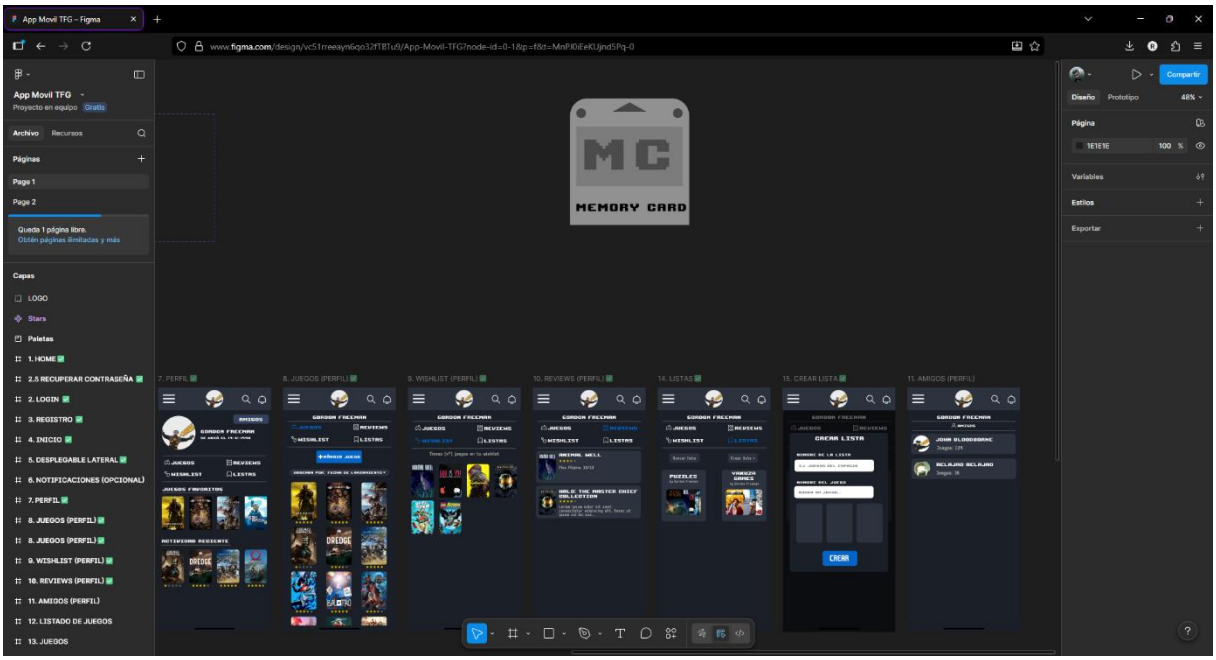
- **Imagen 2.** Tipografías.

Descripción: Definición de la identidad visual diseñada para resaltar elementos gráficos como carátulas y reducir la carga visual.



- **Imagen 3.** Diseños de Interfaz Móvil en Figma I.

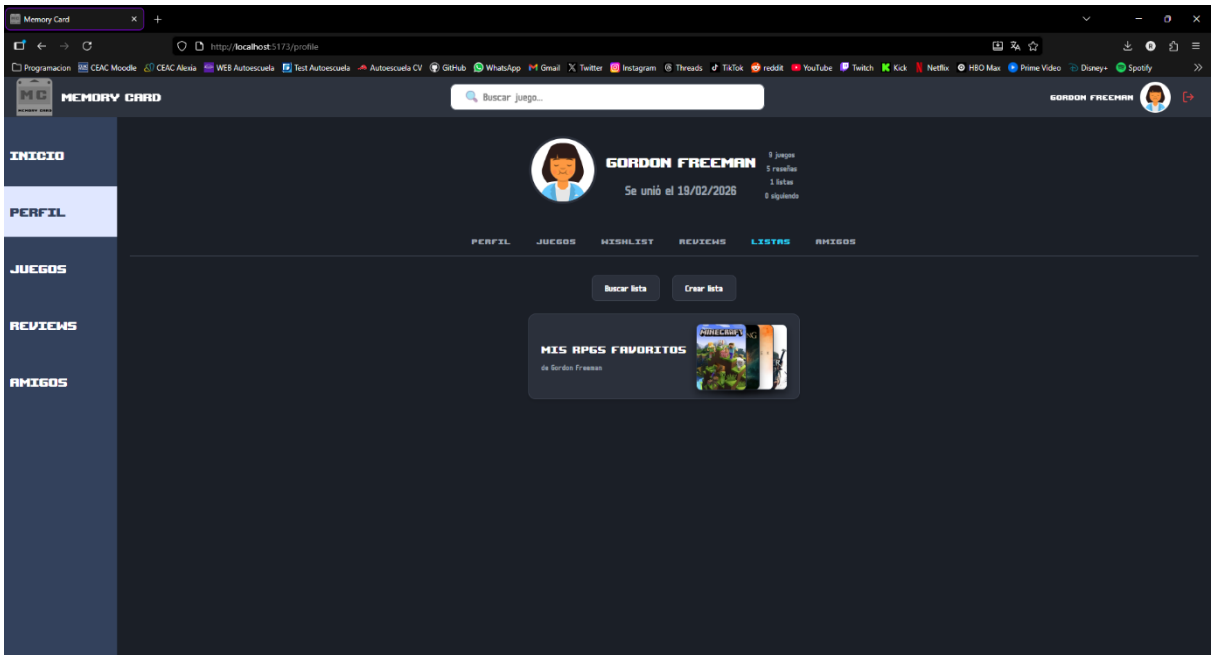
Descripción: Prototipos visuales de los diseños de los apartados de la aplicación móvil (Gestión de usuarios, buscador de juegos y plantilla para las páginas de juegos).



- **Imagen 4.** Diseños de Interfaz Móvil en Figma II.

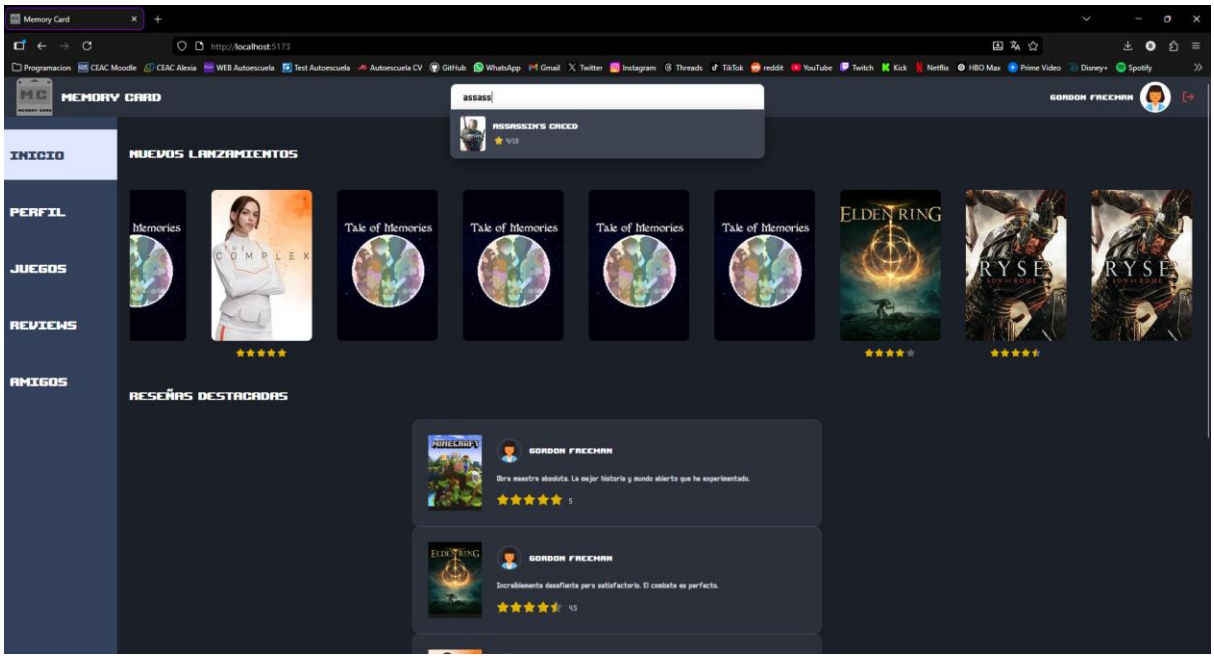
Descripción: Prototipos visuales de los diseños de los apartados de la aplicación móvil (Gestión de listas, plantillas para las páginas de perfiles e icono/logo).

Anexo 4. Implementación y Sincronización



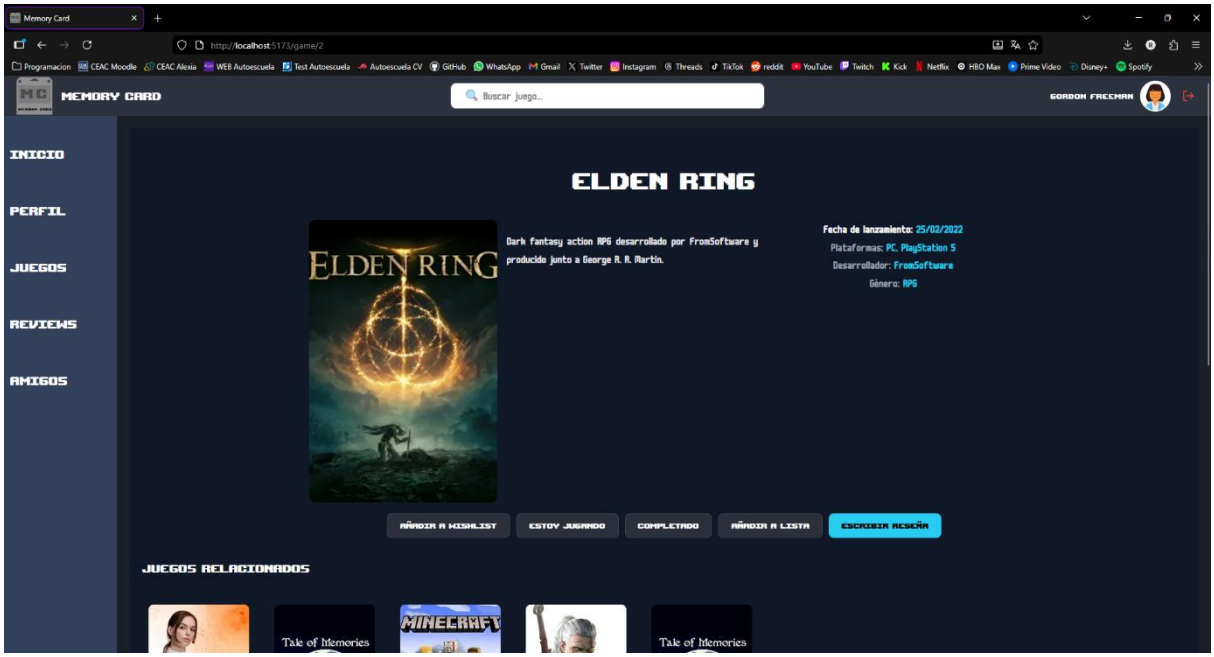
- **Imagen 1. Interfaz del Cliente Web (React) - Perfil.**

Descripción: Captura del Cliente Web en el apartado de listas del perfil.



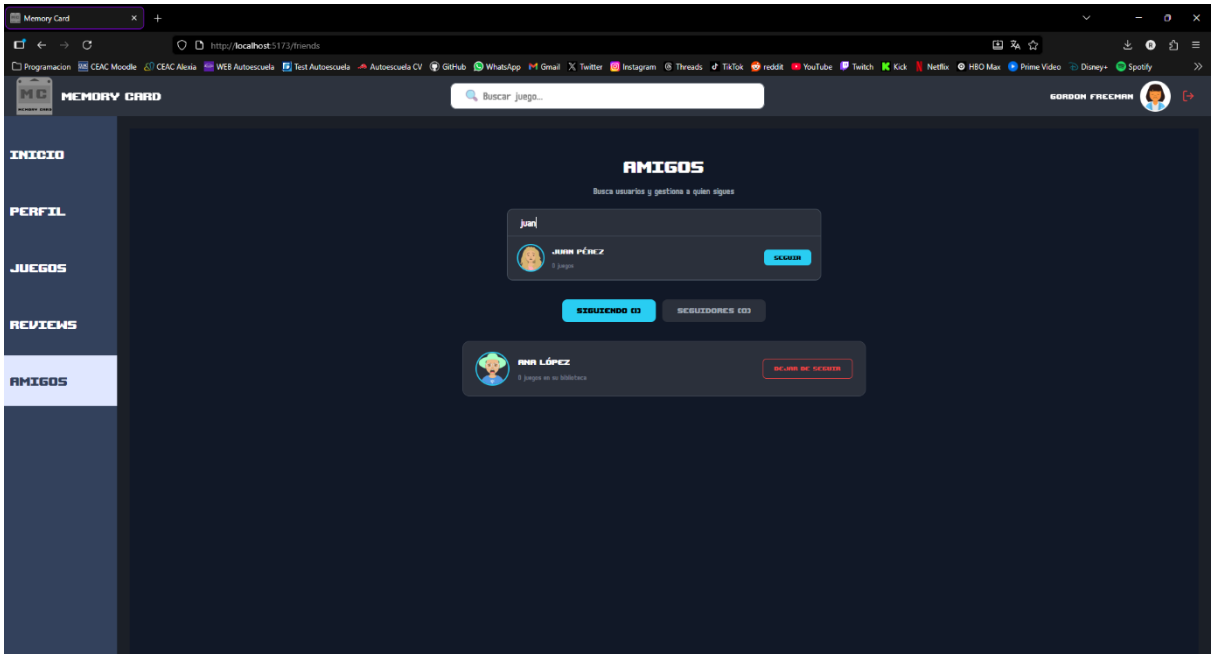
- **Imagen 2. Interfaz del Cliente Web (React) - Inicio.**

Descripción: Captura del Cliente Web en la página principal, mientras se prueba el buscador.



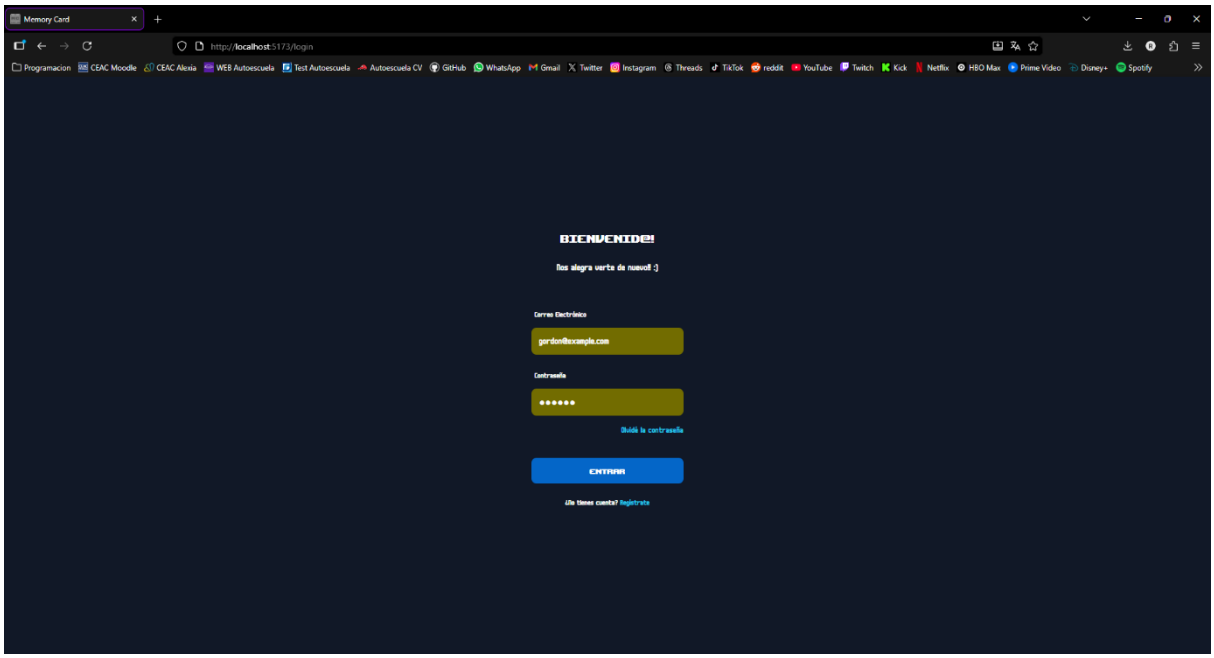
- **Imagen 3. Interfaz del Cliente Web (React) - Juego.**

Descripción: Captura del Cliente Web en la página correspondiente a uno de los juegos.



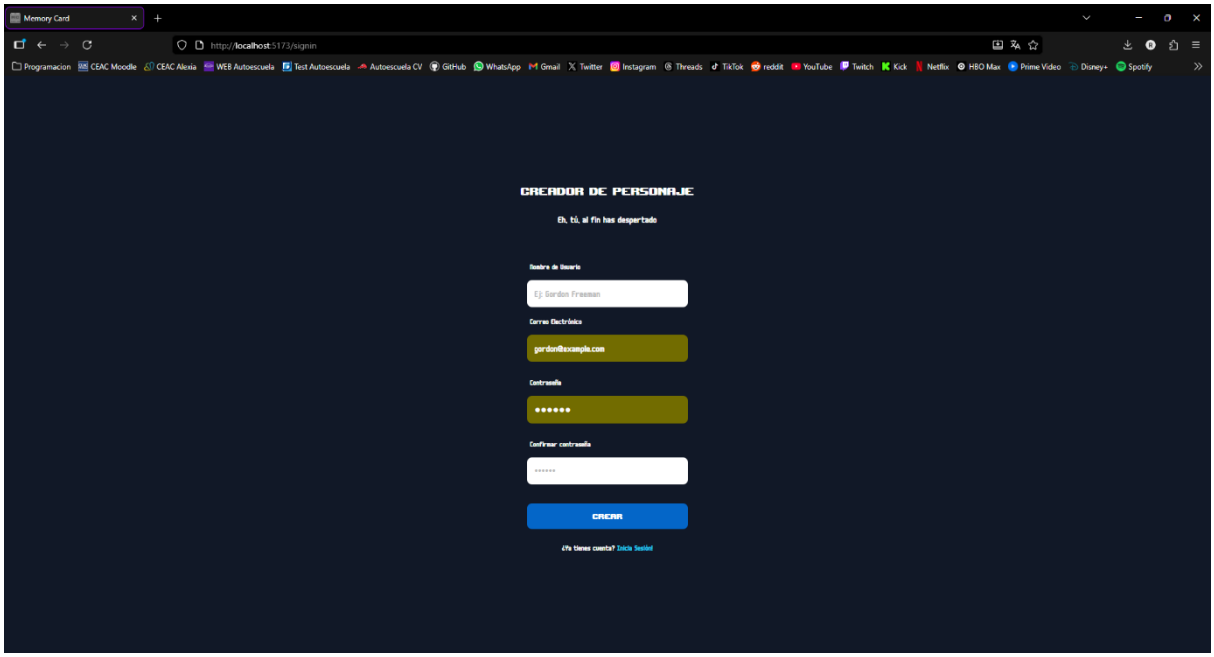
- **Imagen 4. Interfaz del Cliente Web (React) - Amigos.**

Descripción: Captura del Cliente Web en la página de amigos mientras se usa el buscador.



- **Imagen 5. Interfaz del Cliente Web (React) – Inicio de Sesión.**

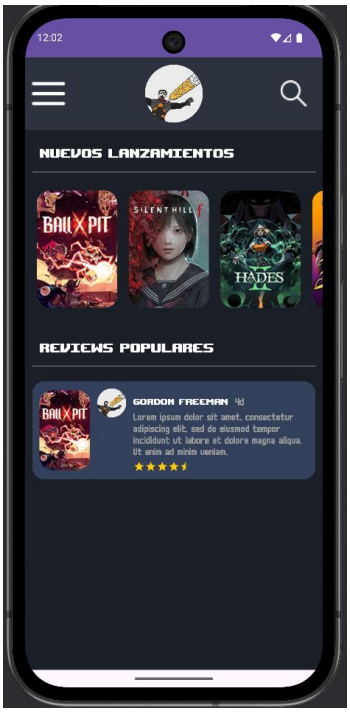
Descripción: Captura del Cliente Web en la página de inicio de sesión.



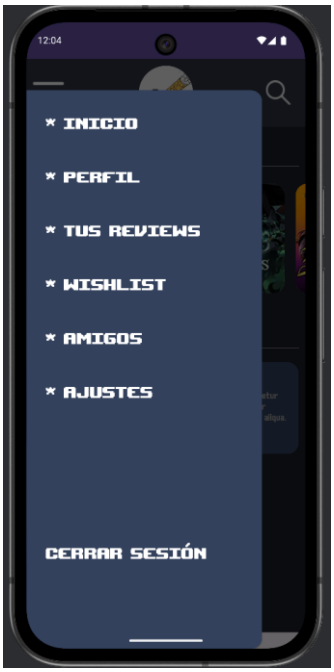
- **Imagen 6. Interfaz del Cliente Web (React) - Registro.**

Descripción: Captura del Cliente Web en la página de registro.

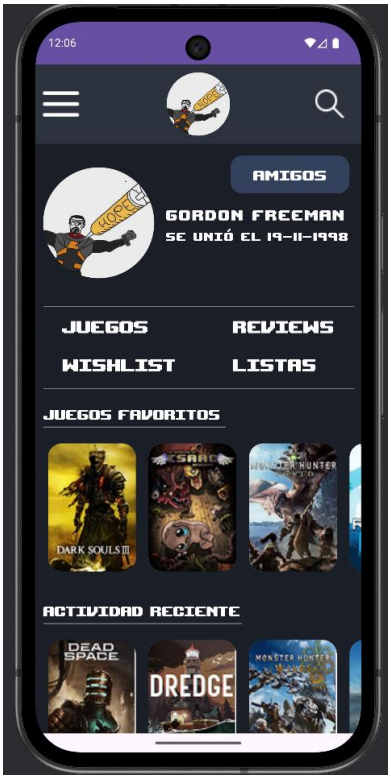
- **Figura 7. Inicio de sesión en la Aplicación Móvil Nativa (Android).**



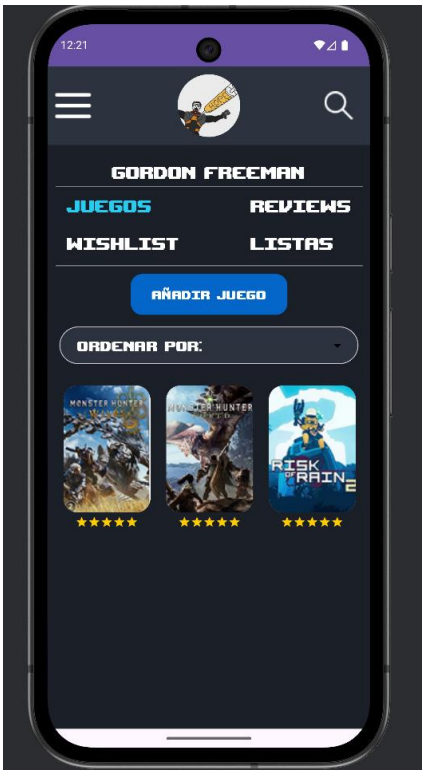
- **Figura 8. Página principal en la Aplicación Móvil Nativa (Android).**



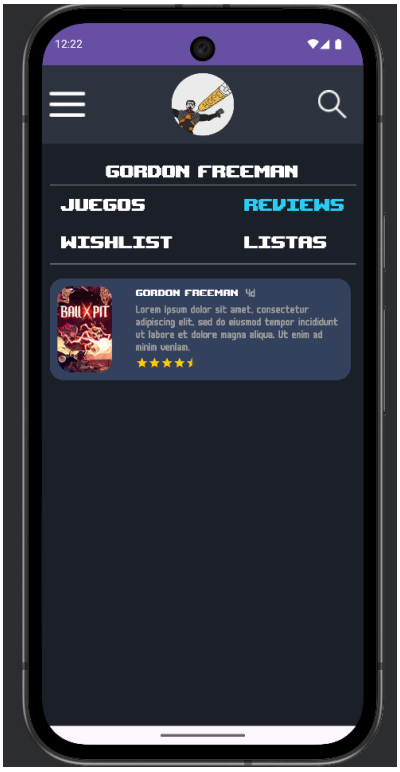
- **Figura 9. Menú desplegable lateral en la Aplicación Móvil Nativa (Android).**



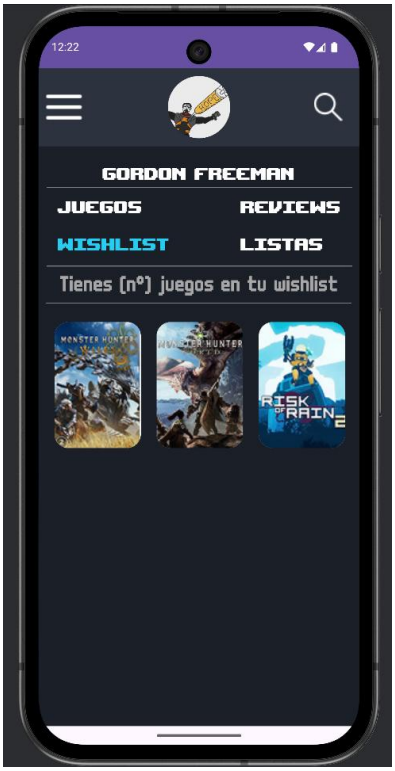
- **Figura 10. Sección principal del perfil en la Aplicación Móvil Nativa (Android).**



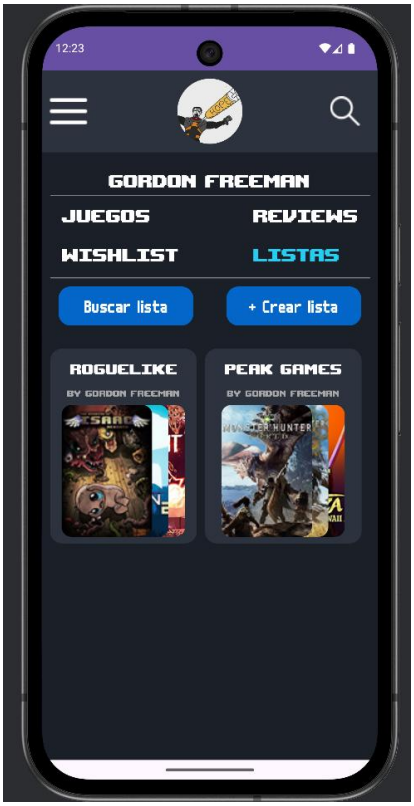
- **Figura 11. Sección de Biblioteca del perfil en la Aplicación Móvil Nativa (Android).**



- **Figura 12. Sección de reviews del perfil en la Aplicación Móvil Nativa (Android).**



- **Figura 13. Sección de Lista de deseos del perfil en la Aplicación Móvil Nativa (Android).**



- **Figura 14. Sección de listas personalizadas del perfil en la Aplicación Móvil Nativa (Android).**